# Chapter 9

## Introduction to Parallel Processing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Basic Uniprocessor Architecture

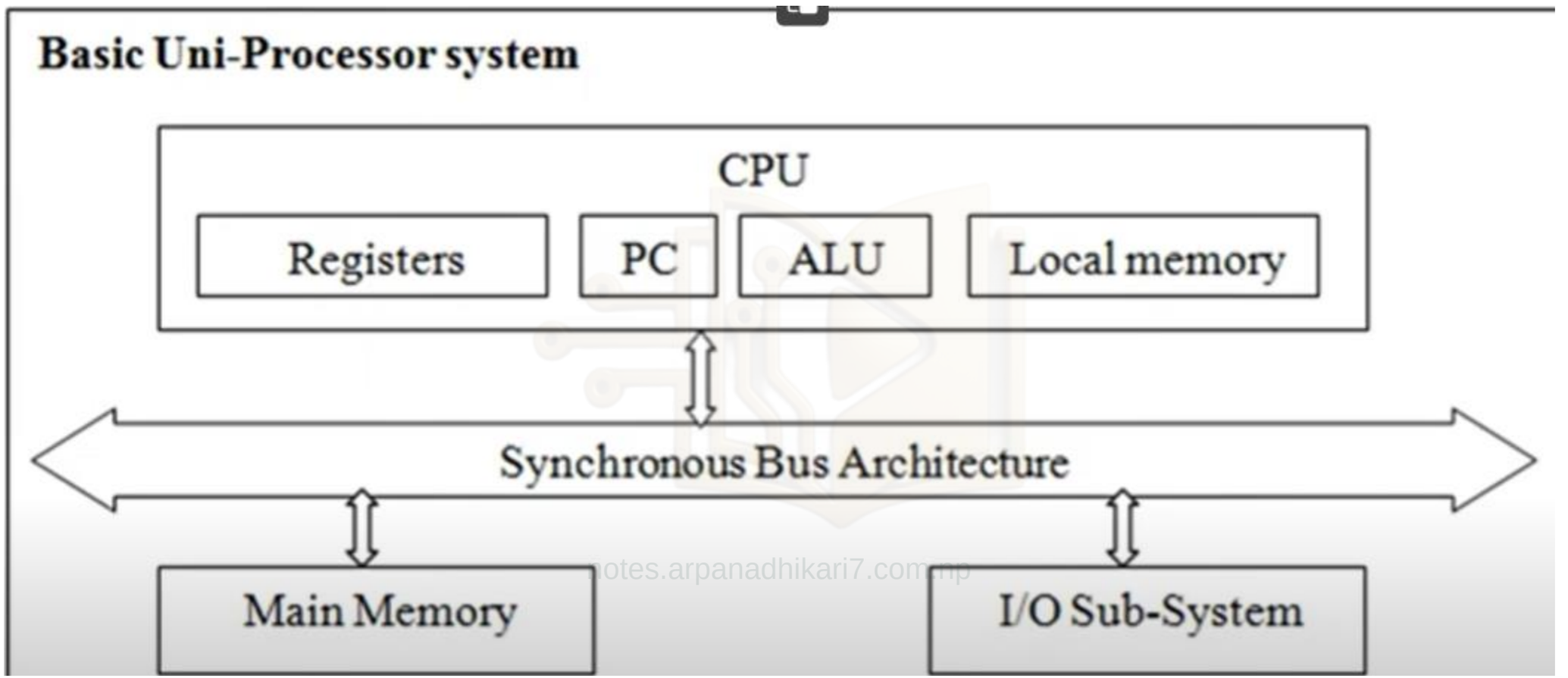
A typical uni-processor computer consists of three major components:

1. **Central Processing Unit (CPU)**
   - Set of General Purpose Registers along with Program Counter.
   - A Special purpose CPU status registers for storing the current state of CPU and program under execution.
   - One Arithmetic and Logic Unit (ALU).
   - One Local Cache Memory.
2. **Main Memory**
3. **Input-Output (I/O) Subsystem**

In addition, there is a **common synchronous bus architecture** for communication between CPU, Main Memory and I/O Subsystem

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np

# Comparison between Sequential and Parallel Computer

- Sequential Computers
  - Are uniprocessor systems ( 1 CPU)
  - Can Execute 1 Instruction at a time
  - Speed is limited
  - It is quite expensive to make single cpu faster
  - Area where it can be used : colleges, labs,
  - Ex: Pentium PC

- Parallel Computers
  - Are Multiprocessor Systems ( many CPU's )
  - Can Execute several Instructions at a time.
  - No limitation on speed
  - Less expensive if we use larger number of fast processors to achieve better performance.
  - Ex : CRAY 1, CRAY-XMP(USA) and PARAM ( India )

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Parallelism in uniprocesor System

- Parallel processing mechanisms to achieve parallelism in uniprocessor system are :
  - **Multiple function units**
  - **Parallelism and pipelining within CPU**
  - **Overlapped CPU and i/o operations**
  - **Use of hierarchical memory system**
  - **Multiprogramming and time sharing**

notes.arpanadhikari7.com.np

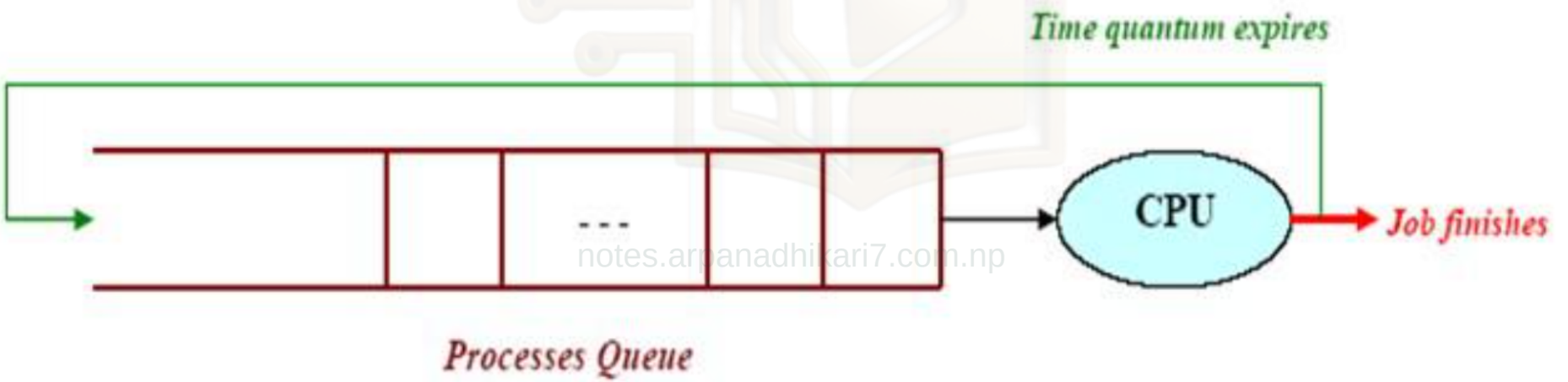
Available on notes.arpanadhikari7.com.np

# Parallelism in uniprocesor System

## 1. Multiprogramming & Timesharing

a. In *multiprogramming* several processes reside in main memory and CPU switches from one process (say $P_1$) to another (say P2) when the currently running process ($P_1$) blocks for an I/O operation. I/O operation for $P_1$ is handled by a DMA unit while the CPU runs $P_2$.

b. In *timesharing* processes are assigned slices of CPU's time. The CPU executes the processes in the round robin fashion as below.




notes.arpanadhikari7.com.np

- It appears that every user (process) has its own processor. (multiple virtual processors)
- Averts the monopoly of a single (computation intensive) process as in pure multiprogramming

Available on notes.arpanadhikari7.com.np

# Parallelism in uniprocesor System

## 2. Multiplicity of Functional Units

Use of multiple functional units like multiple adders, multipliers or even multiple ALUs to provide concurrency is not a new idea in uniprocessor environment. It has been around for decades.

## 3. Harvard Architecture

a. This provides separate memory units for instructions and data. This effectively doubles the memory bandwidth saving CPU's time. E.g. is split cache in which instructions are kept in I-cache and data in D-cache

b. In contrast when instructions and data are kept in the same memory, the architecture is called Princeton Architecture. E.g. are unified cache, main memory, etc.

notes.arpanadhikari7.com.np

## 4. Memory Hierarchy

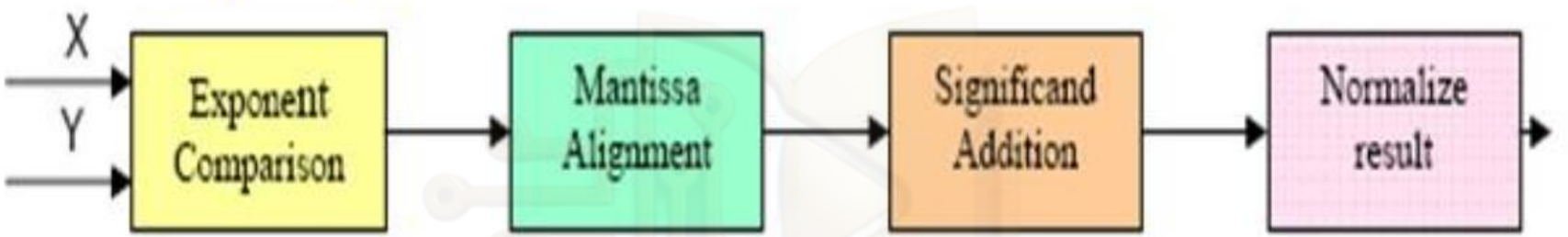
A parallel processing mechanism supported by memory hierarchy is the simultaneous transfer of instructions/data between (CPU, cache) and (MM, secondary memory)

Available on notes.arpanadhikari7.com.np

# Parallelism in uniprocesor System

5. Pipelining

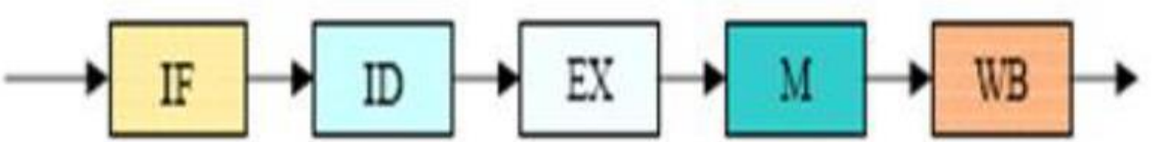
a. Arithmetic Pipelining




*4-stage FP adder*

b. Instruction Pipelining

notes.arpanadhikari7.com.np




*5-stage instruction pipeline*

Available on notes.arpanadhikari7.com.np

# Multiprocessor system

This system achieves asynchronous parallelism through a set of interactive processors with shared resources (memories ,databases etc.) .

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

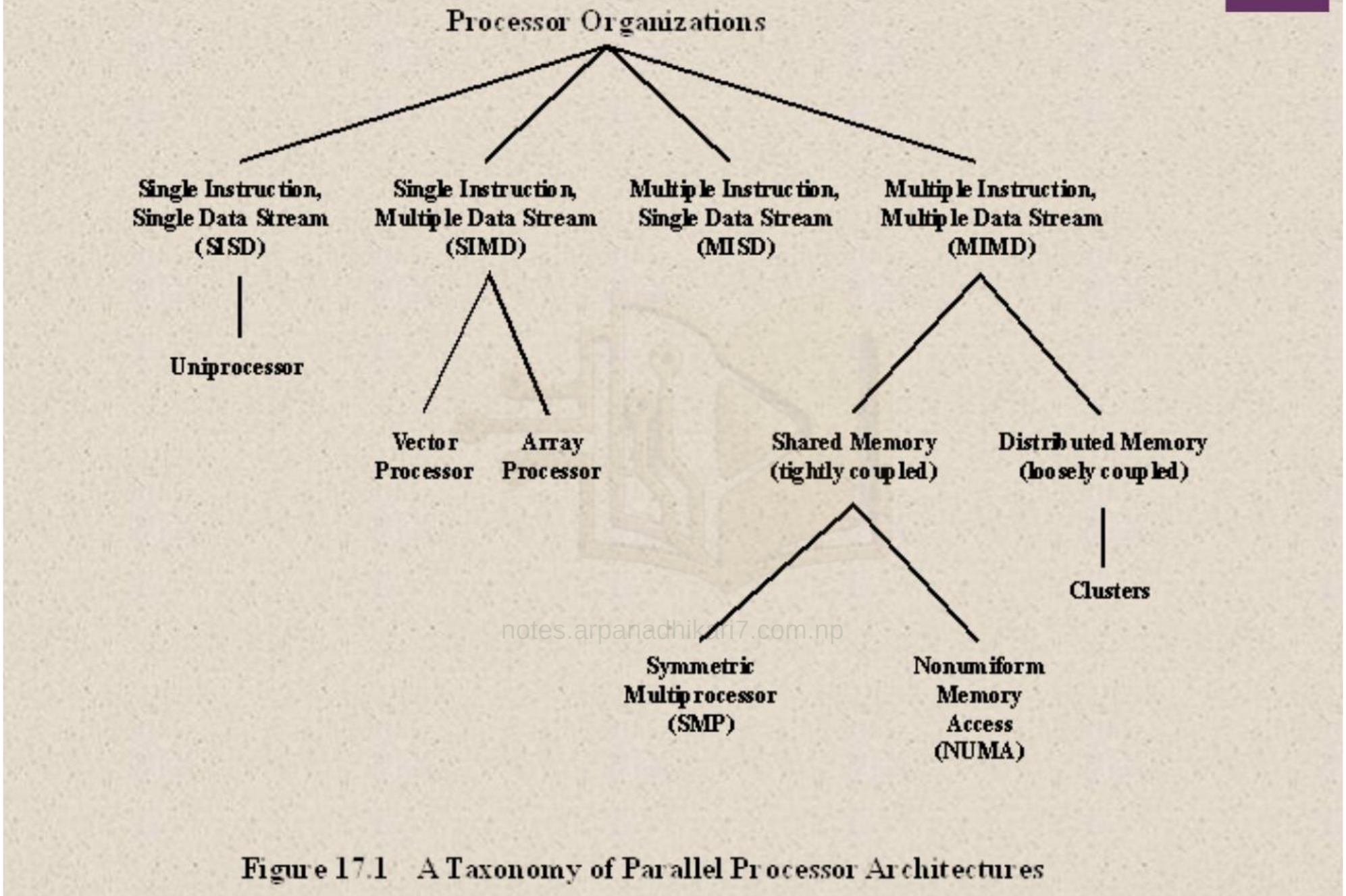



Figure 17.1 A Taxonomy of Parallel Processor Architectures

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

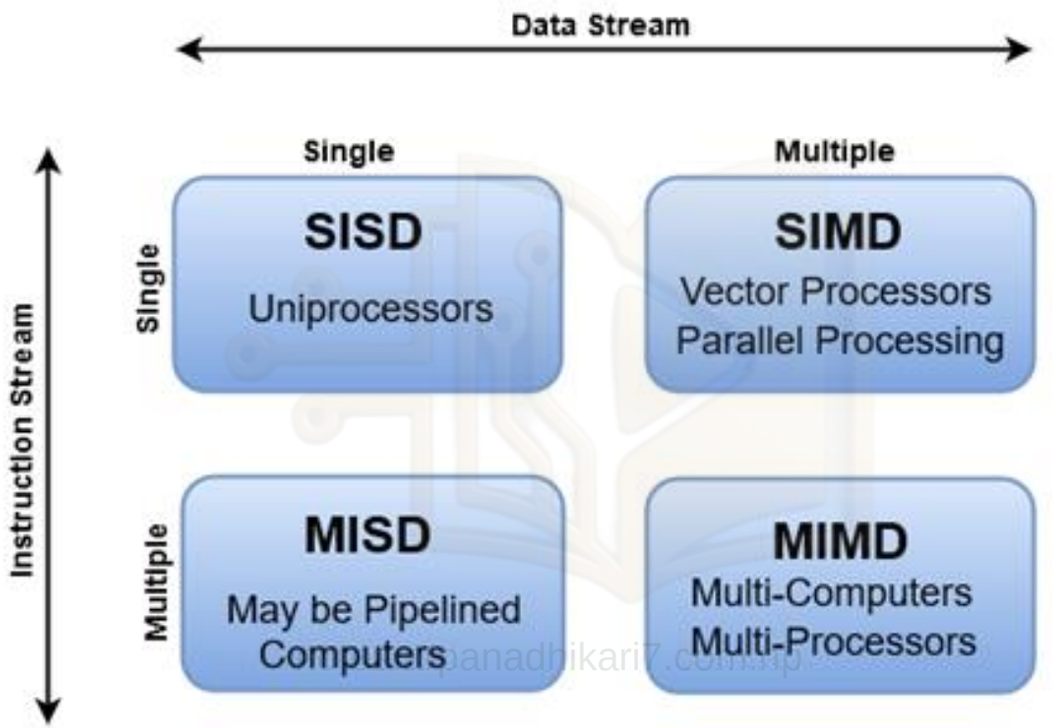
# Multiple Processor Organization

- Single instruction, single data (SISD) stream
  - Single processor executes a single instruction stream to operate on data stored in a single memory
  - Uniprocessors fall into this category
- Single instruction, multiple data (SIMD) stream
  - A single machine instruction controls the simultaneous execution of a number of processing elements on a lockstep basis
  - Vector and array processors fall into this category
- Multiple instruction, single data (MISD) stream
  - A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence
  - Not commercially implemented
- Multiple instruction, multiple data (MIMD) stream
  - A set of processors simultaneously execute different instruction sequences on different data sets
  - SMPs, clusters and NUMA systems fit this category

notes.arpanadhikari7.com.np

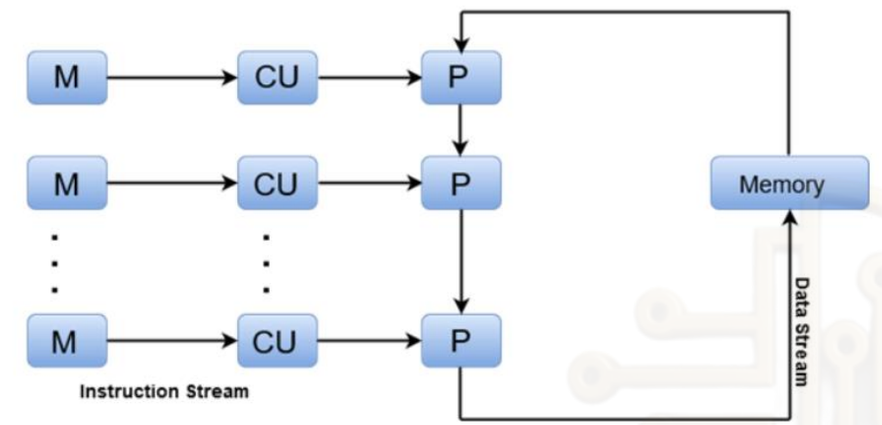
Available on notes.arpanadhikari7.com.np

## Flynn's Classification of Computers




Available on notes.arpanadhikari7.com.np

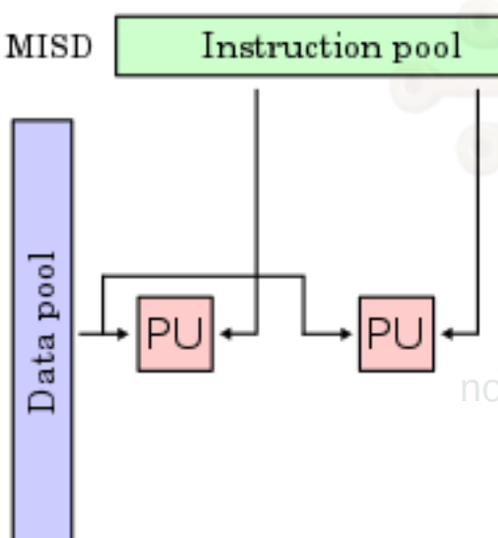
**MISD:**




In MISD, there are multiple processing elements that have their own control unit and local memory which enables each processor to handle instructions independently. To speed up the problem-solving process, the problem is divided into sub-problems and each sub-problem has its own set of programs or instruction streams. Different instruction streams are fed to the control unit of each processor and in turn, the control units send the instruction streams to the processors. Each processor therefore operates on the data independently and processes the instruction streams asynchronously. In most cases, the output of one processor becomes an input of the next processor which means the processors are executing different programs or solving different sub-problems of the main problem.

Available on notes.arpanadhikari7.com.np

**Leiserson** are an example of MISD architecture. In a typical systolic array, parallel input data flows through a network of hard-wired processor nodes, resembling the human brain which combine, process, merge or sort the input data into a derived result.




notes.arpanadhikari7.com.np

## MIMD - Overview

- General purpose processors
- Each can process all instructions necessary
- Further classified by method of processor communication

## Tightly Coupled - NUMA

- Nonuniform memory access
- Access times to different regions of memory may differ

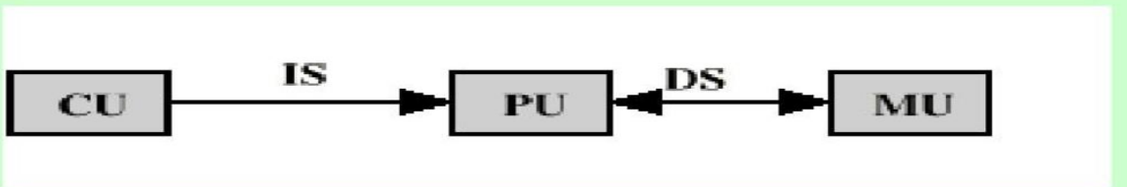
## Tightly Coupled - SMP

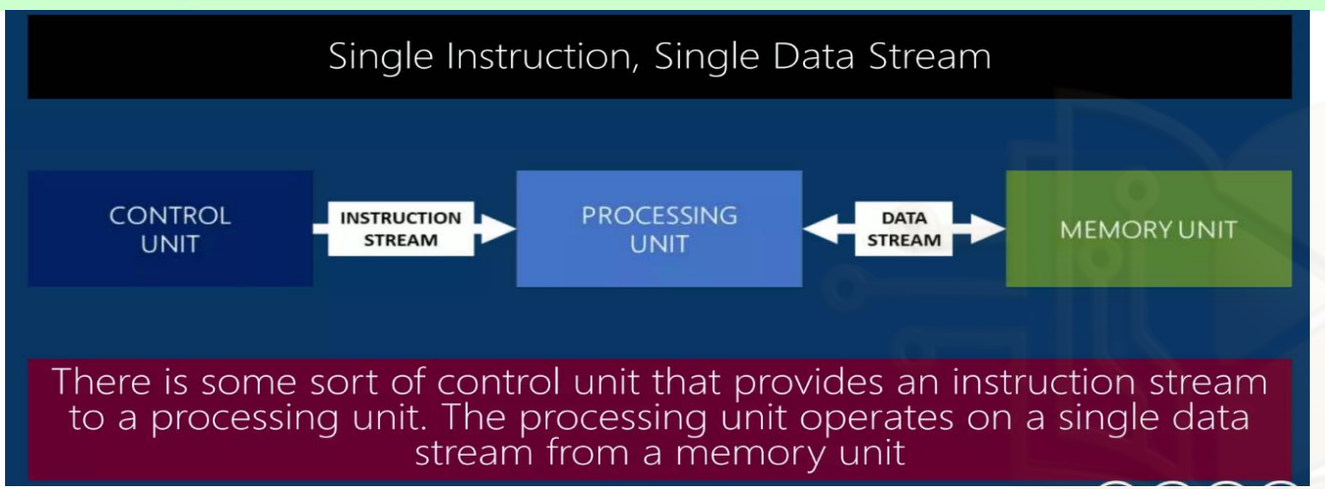
- Processors share memory
- Communicate via that shared memory
- Symmetric Multiprocessor (SMP)
  - —Share single memory or pool
  - —Shared bus to access memory
  - —Memory access time to given area of memory is approximately the same for each processor

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Parallel Organizations - SISD



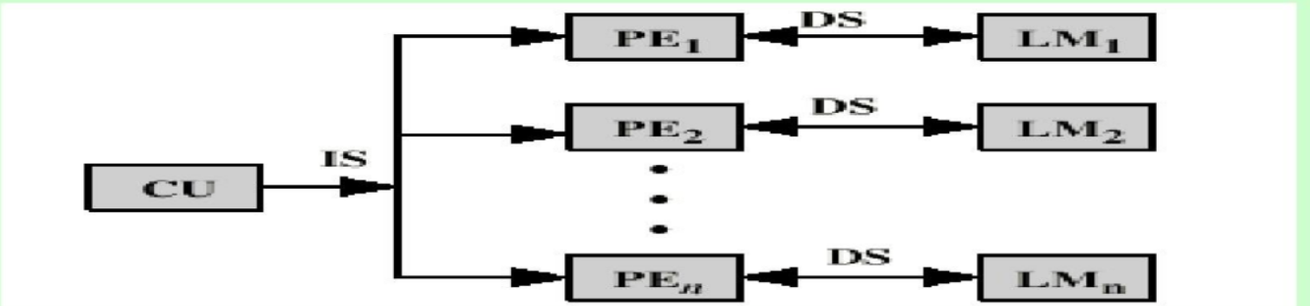



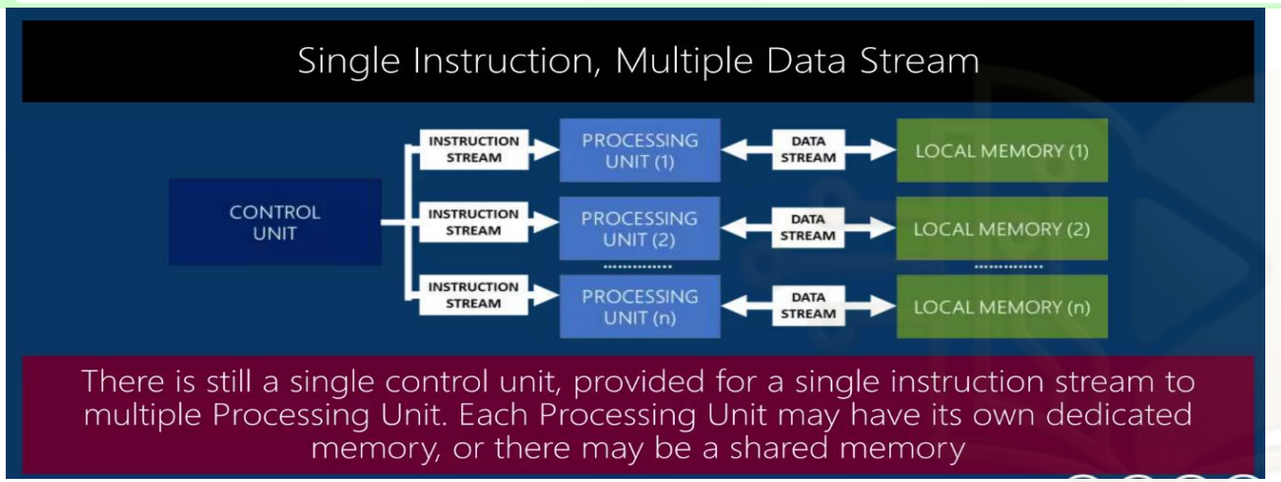

Dominant representative SISD systems are **IBM PC, Macintosh, and workstations**.

**Pipelined processors and superscalar processors** are common examples found in most modern SISD computers.

notes.arpanadhikari7.com.np

# Parallel Organizations - SIMD

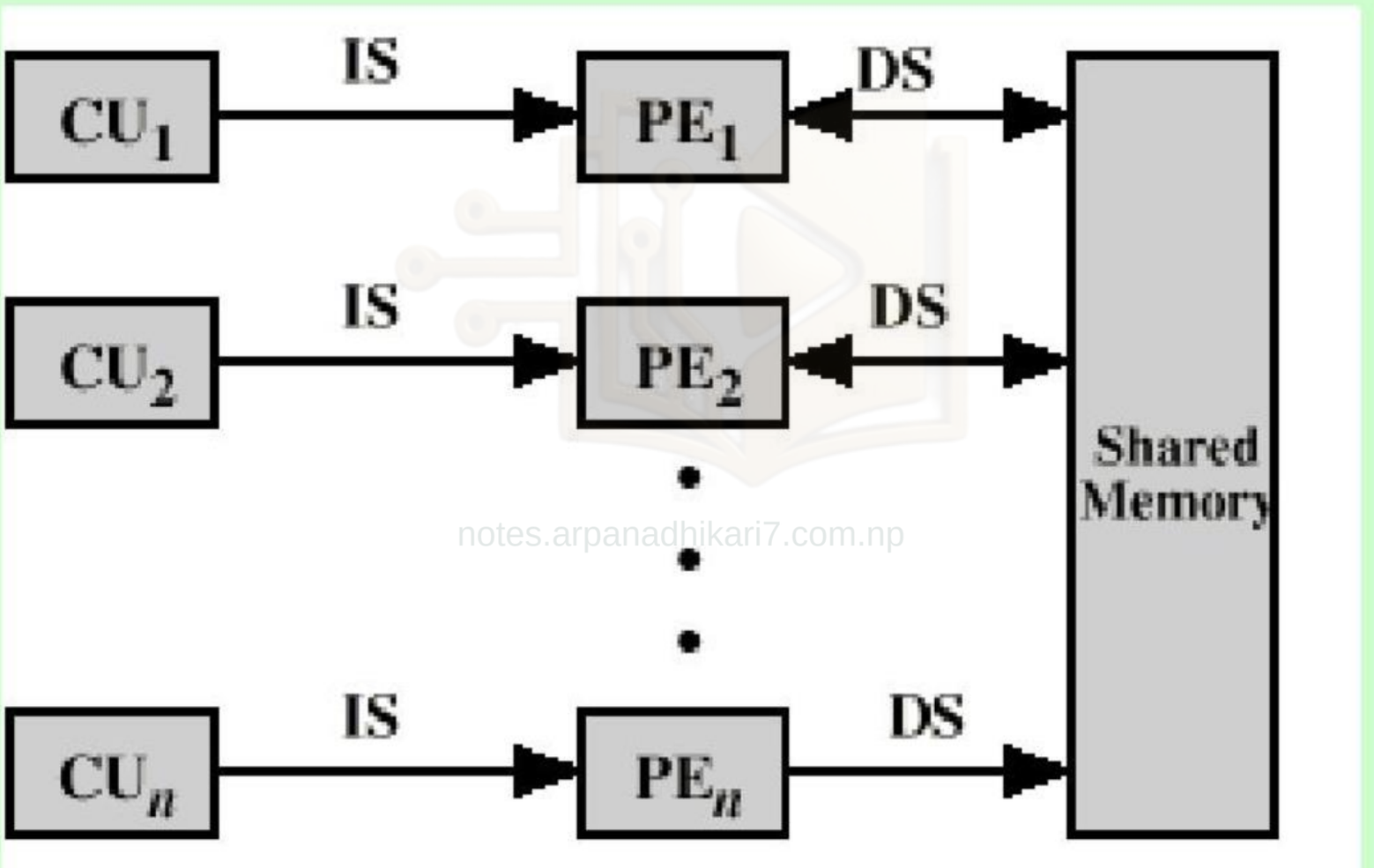



Today, most commodity processors are based on hardware architectures that feature SIMD vector instructions. **Intel MMX/SSE/VX/AVX-512, IBM Power AltiVec and Cell SPU, and ARM NEON**

notes.arpanadhikari7.com.np

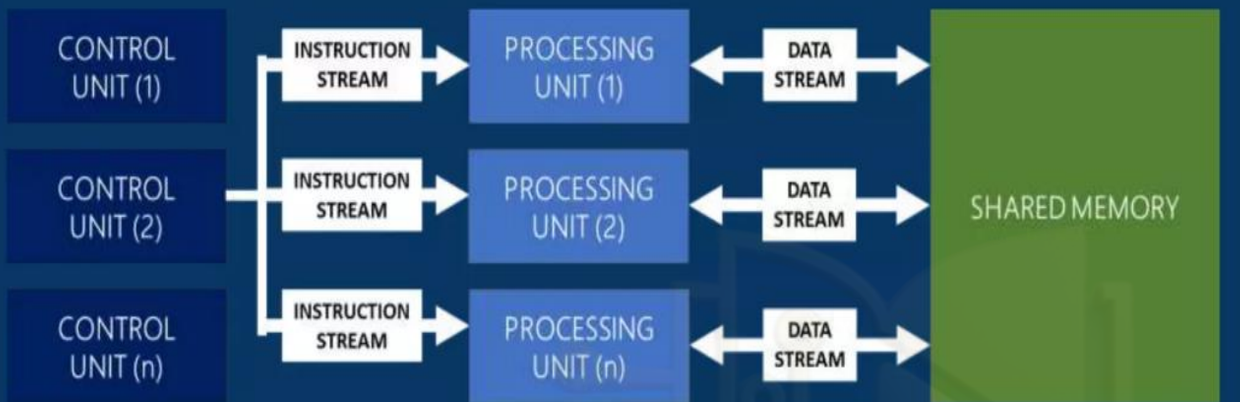
# Parallel Organizations - MIMD Shared Memory




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Multiple Instruction, Multiple Data Stream (Shared Memory)



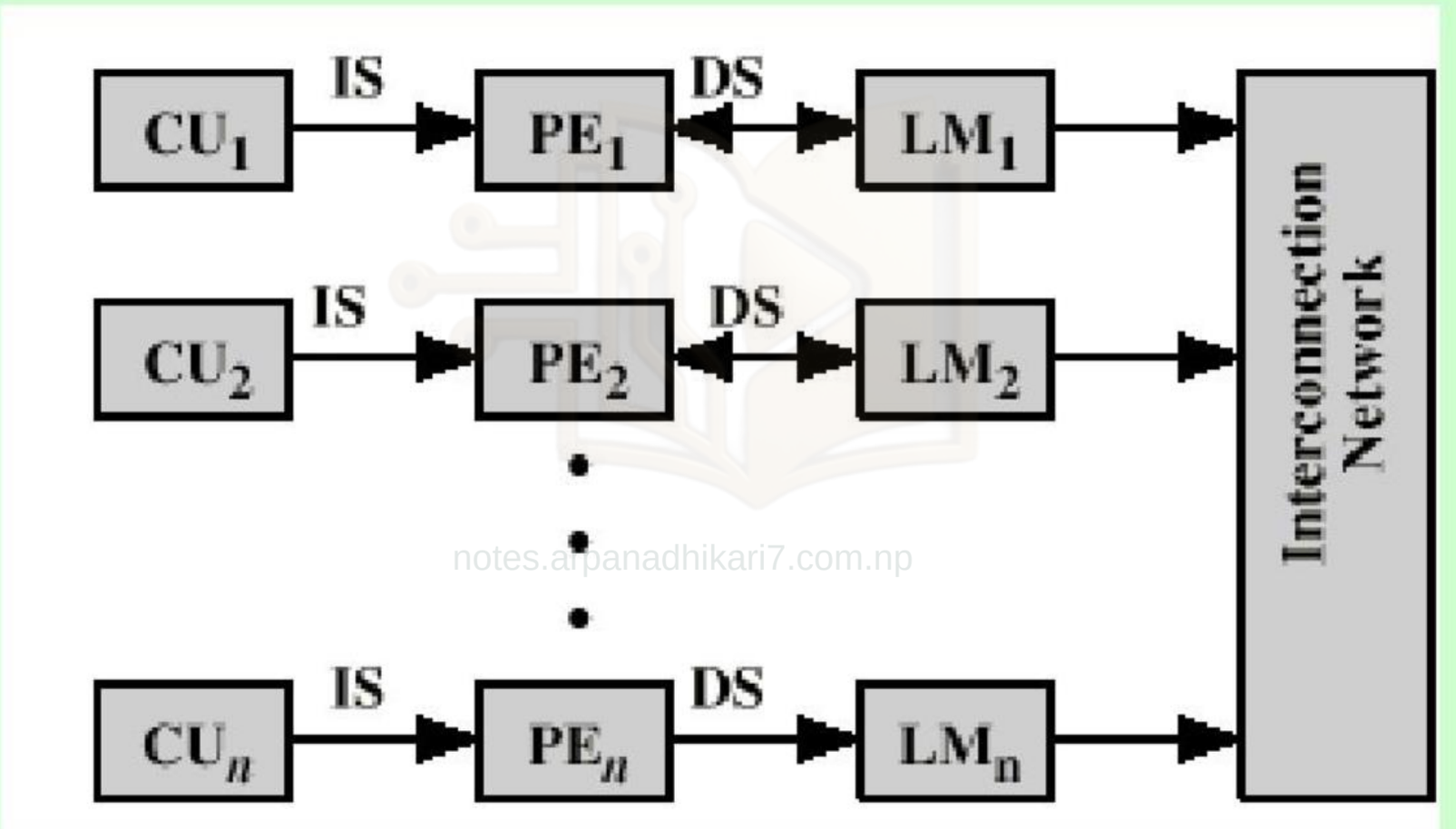

In MIMD, There are multiple control units, each feeding a separate instruction stream to its own Processing unit. The MIMD may be a shared-memory multiprocessor or a distributed- memory multicomputer.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Parallel Organizations - MIMD Distributed Memory

An example of MIMD system is **Intel Xeon Phi**




notes.arpanadhikari7.com.np

# Symmetric Multiprocessors

- A stand alone computer with the following characteristics
  - **Two or more similar processors of comparable capacity**
  - **Processors share same memory and I/O**
  - **Processors are connected by a bus or other internal connection**
  - **Memory access time is approximately the same for each processor**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Symmetric Multiprocessors (continued)

- All processors share access to I/O
  - Either through same channels or different channels giving paths to same devices
- All processors can perform the same functions (hence symmetric)
- System controlled by integrated operating system
  - providing interaction between processors
  - Interaction at job, task, file and data element levels
  - Schedules processes or threads across all processors

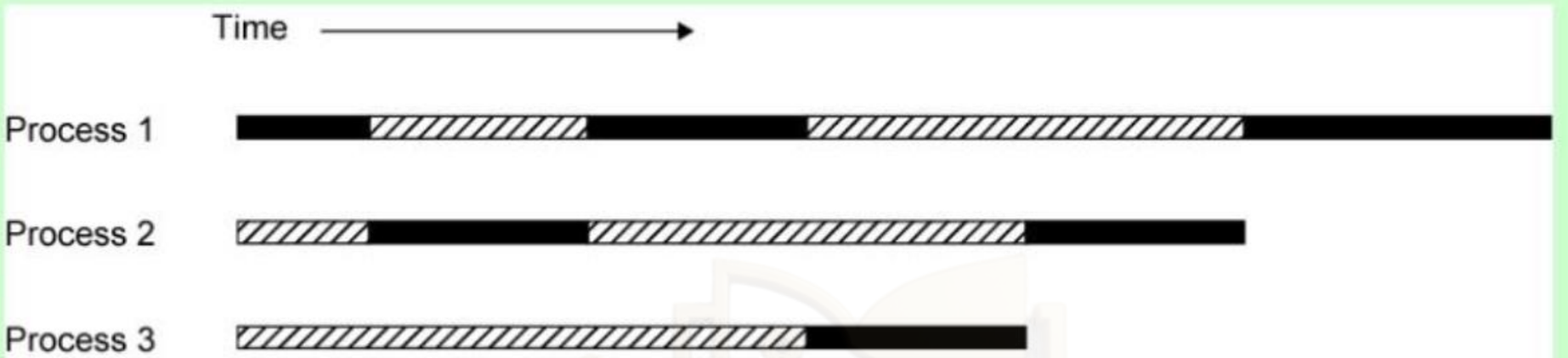
( SMP interaction can be as small as the data element level)

(Cluster systems interact at the message or file level)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Multiprogramming and Multiprocessing




(a) Interleaving (multiprogramming, one processor)




(b) Interleaving and overlapping (multiprocessing; multiple processors)

notes.arpanadhikari7.com.np

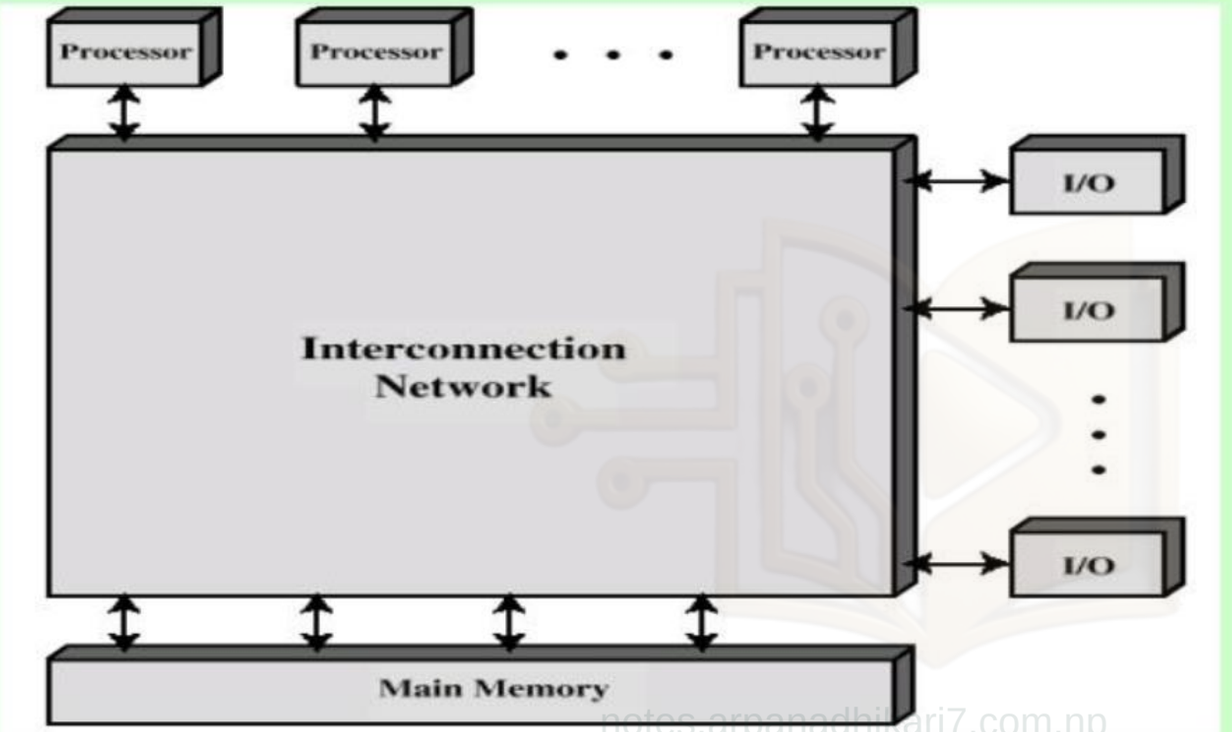
Available on notes.arpanadhikari7.com.np

# SMP Advantages

- **Performance**
  - If some work can be done in parallel, system yields greater performance
- **Availability**
  - Since all processors can perform the same functions, failure of a single processor does not halt the system
- **Incremental growth**
  - User can enhance performance by adding additional processors
- **Scaling**
  - Vendors can offer range of products based on number of processors configured in the system

Available on notes.arpanadhikari7.com.np

# Block Diagram of Tightly Coupled Multiprocessor




Existence of multiple processors is transparent to the user; the O/S schedules threads &/or processes on individual processors and manages the synchronization among processors

Each processor is self-contained, i.e., control unit, ALU, registers, cache, access to shared main memory and shared I/O devices via interconnection mechanism and possibly private main memory and I/O channels separate from the shared resources.

Processors communicate via messages and status information placed in common data areas of shared main memory.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

## Organization Classification of Multiple Processors

- Time shared or common bus
- Multiport memory
- Central control unit

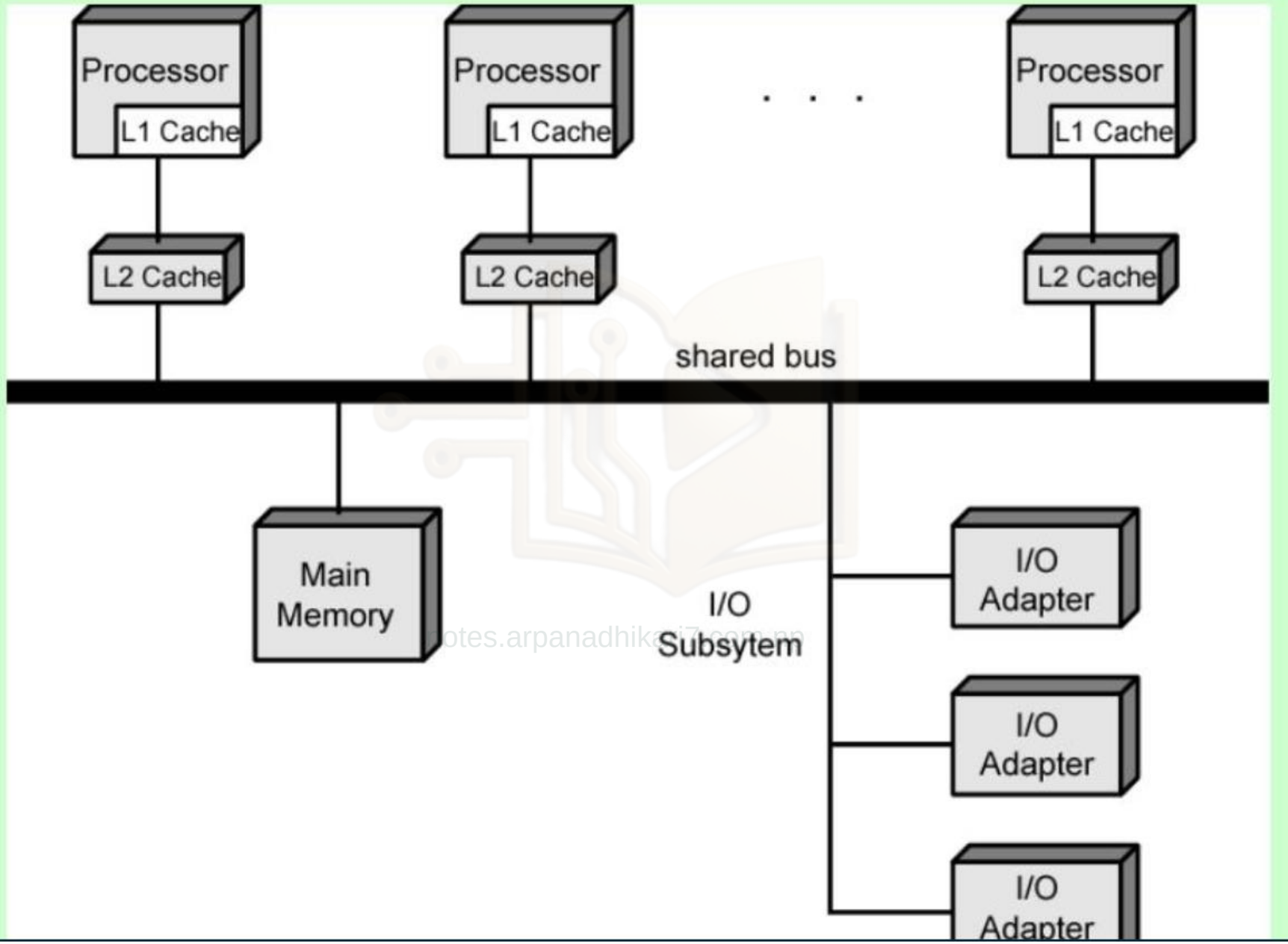
## Time Shared Bus

- Simplest form
- Structure and interface similar to single processor system
- Following features provided
  - —Addressing - distinguish modules on bus
  - —Arbitration - any module can be temporary master
  - —Time sharing - if one module has the bus, others must wait and may have to suspend
- Now have multiple processors as well as multiple I/O modules

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Symmetric Multiprocessor Organization




Available on notes.arpanadhikari7.com.np

## Time Share Bus - Advantages

- Simplicity
  - **interface, addressing, arbitration, and time-sharing logic of each processor remains unchanged**
- Flexibility
  - **Easy to attach more processors**
- Reliability
  - **Failure of any attached device should not cause the failure of the entire system**

## Time Share Bus - Disadvantage

- Performance limited by bus cycle time
- Each processor should have local cache
  - Reduce number of bus accesses
  - L1, L2, L3 cache
- Leads to problems with cache coherence
  - Solved in hardware - see section 17.4

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Cache Coherence Problem

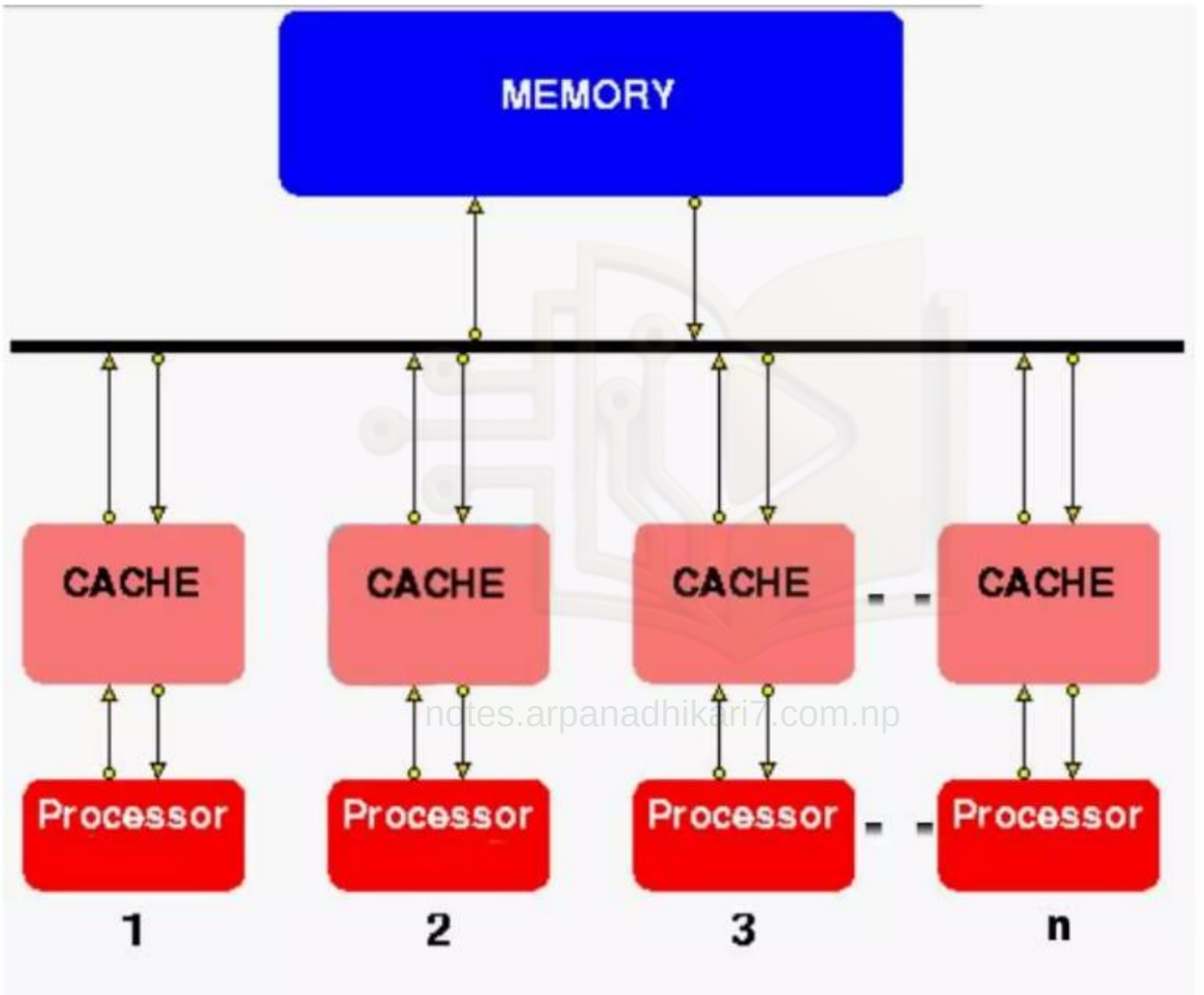
- Caches in multiprocessing environment introduce the Cache Coherence problem.

- When multiple processors maintain locally cached copies of a unique shared memory location, any local modification of the location can result in a globally inconsistent view of memory. This is called Cache Coherence Problem.

notes.arpanadhikari7.com.np

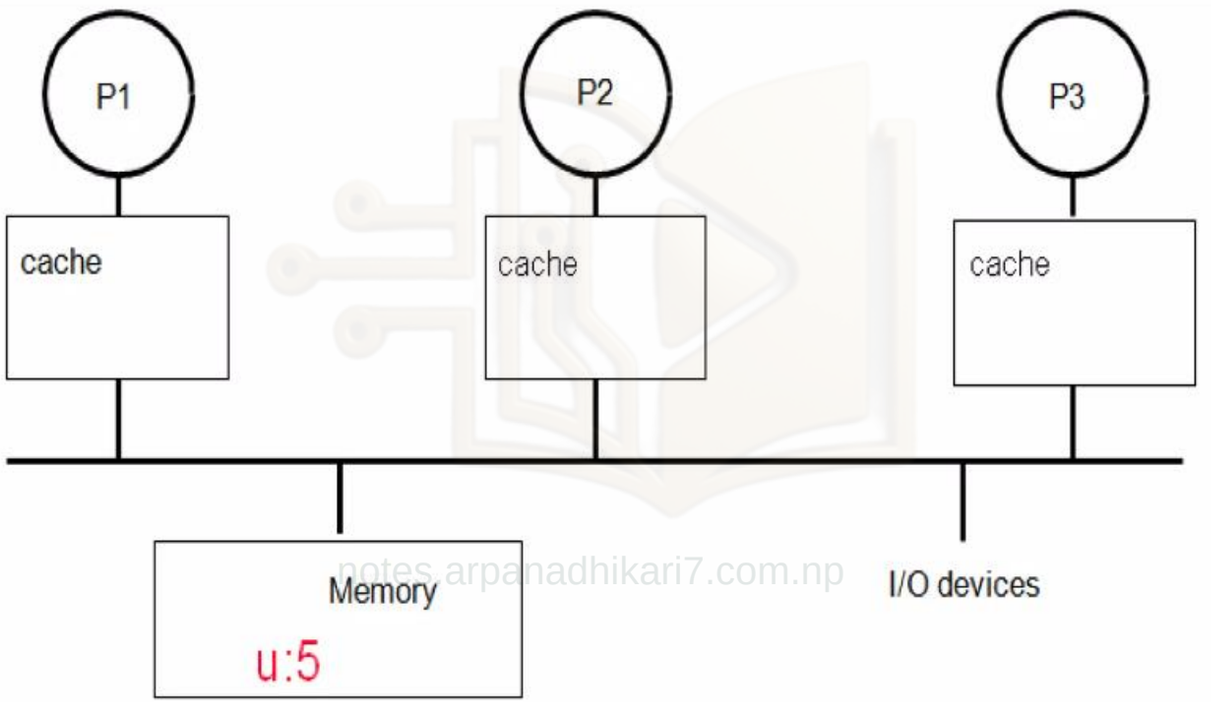
Available on notes.arpanadhikari7.com.np

# Cache Coherence




notes.arpanadhikari7.com.np

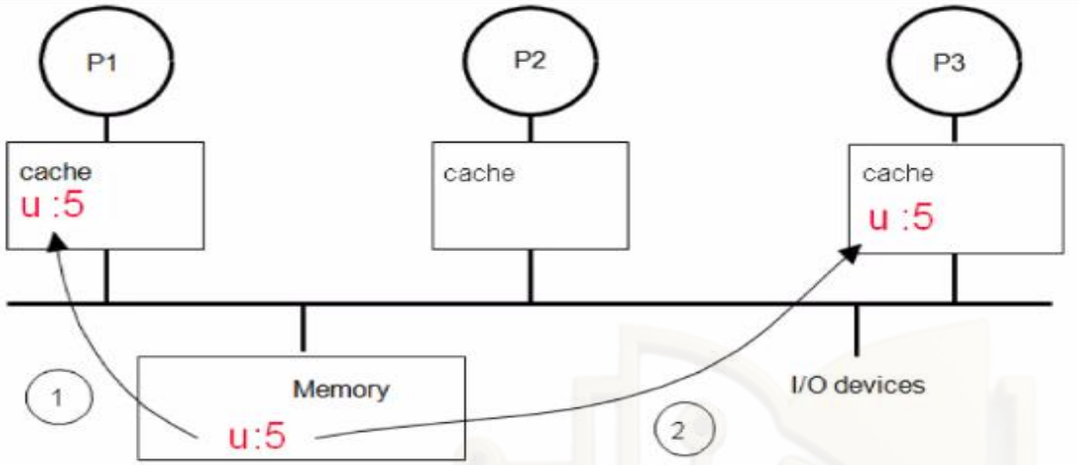
# Example on Cache Coherence Problem

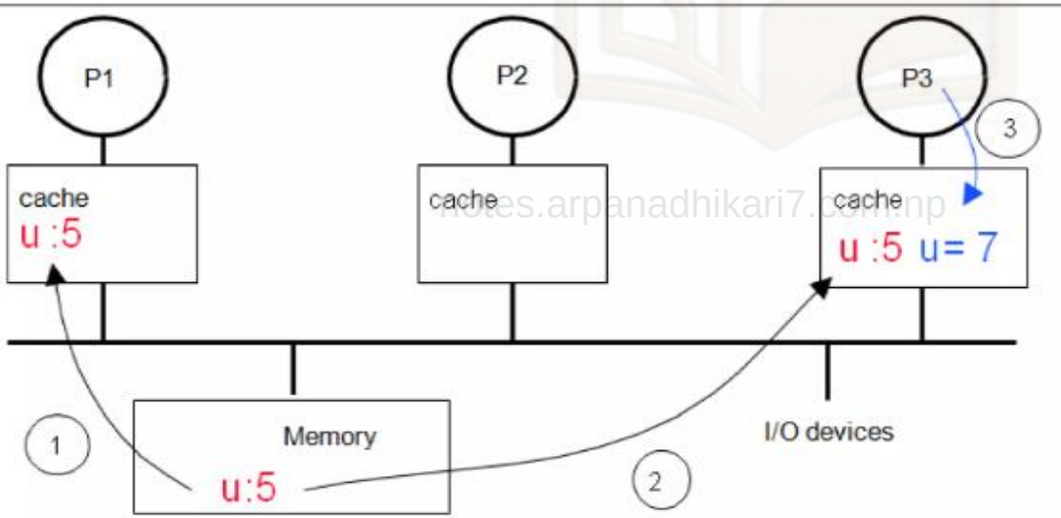



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Example on Cache Coherence Problem

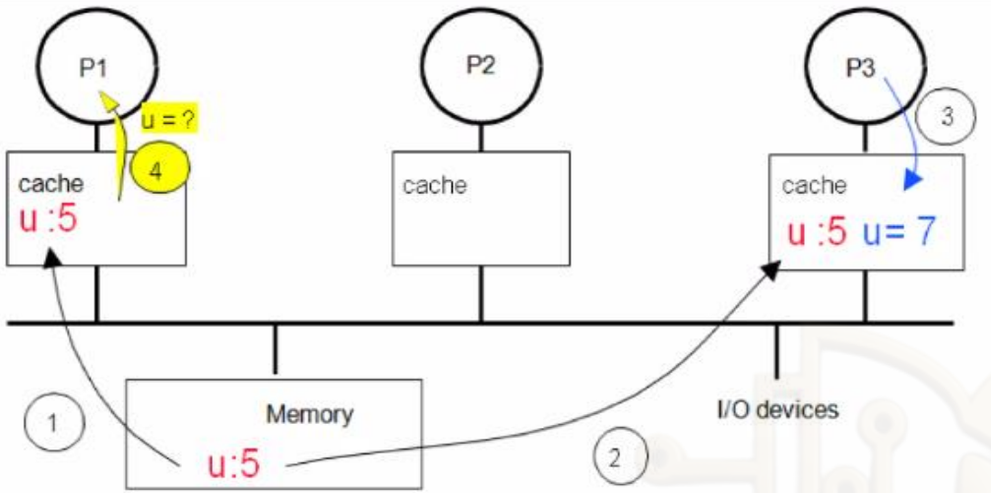



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Example on Cache Coherence Problem




Unlike uniprocessor systems, however, multiprocessors have individual caches for each processor.

This can lead to problems when two or more caches hold the value of the same memory location simultaneously.

As one processor stores a value to that location in its cache, the other cache will have an invalid value in its location.

Using a write-through cache will not resolve this problem, since it would update main memory but not the other caches.

In addition, the extra writes to main memory would decrease system performance.

This is the **cache coherence** problem.

Available on notes.arpanadhikari7.com.np

- To illustrate this cache coherence problem, consider a multiprocessor system with four processors, each of which has a write-back cache.

Cache values illustrating the cache coherence problem

| Action | Cache 0 | Cache 1 | Cache 2 | Cache 3 |
|---|---|---|---|---|
| Initial | 1234: 56 | 1234: 56 | 1234: 56 | 1234: 56 |
| Processor 0 updates 1234H | 1234: 78 | 1234: 56 | 1234: 56 | 1234: 56 |
| Processor 3 reads 1234H | | | | Reads 56H instead of 78H |

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Cache Coherency Solutions

- **Snoopy Bus Protocol :**

  A bus is a good medium for processors to observe ongoing memory transactions. If a bus transaction creates inconsistency of a locally cached object, cache controller can take actions to invalidate the local copy. Such a protocol is called Snoopy Bus Protocol

- **Directory Schemes :**

  These protocols use a directory structure to keep track of which processors are sharing each block of data.

One popular solution to the cache coherence problem is called **snooping**.

In snooping, each cache(sometimes called snoopy cache) monitors memory activity on the system bus.

Whenever it encounters a memory access(by another processor) to a location that it currently holds, it takes appropriate action.
=> If the request is memory read, and the contents in its cache are the same as those in main memory, it simply notes that another cache also contains this data.
=> If the request is memory read, and the contents in its cache are different than those in main memory, the processor must have written data to the cache that has not yet been written back to main memory.
The cache intercepts the memory read request, sending its data to both main memory and the cache which requested the data.
=> If the request is the memory write, it simply marks its own data as invalid, essentially removing it from the cache.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Hardware solution scheme is the **cache directory.**

A directory controller is integrated with the main memory; it maintains a cache directory in main memory, which contains information on the contents of local caches.

All cache writes are also sent to the directory controller so that it can update the cache directory.

When the processor writes data to its cache, the directory controller checks to see which other caches also have that data.

It invalidates that data in those caches by marking its locations as empty.

notes.arpanadhikari7.com.np

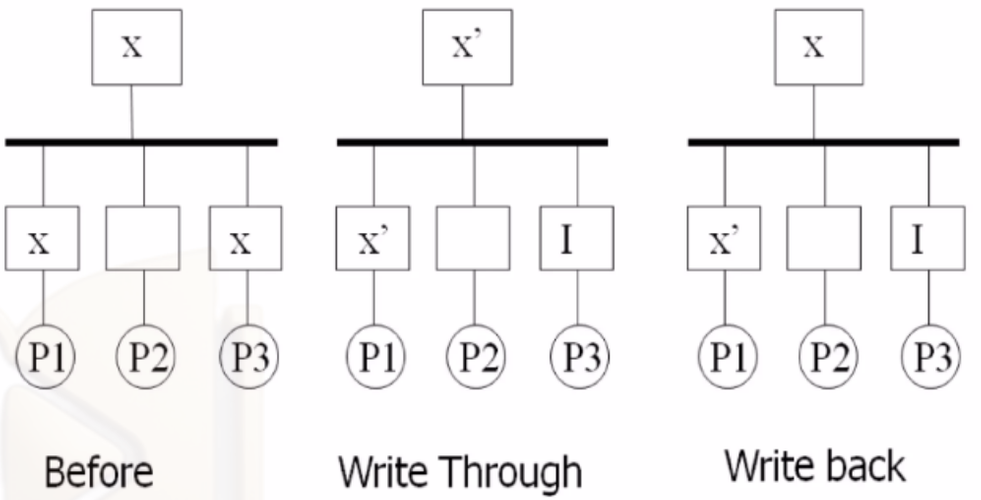
Available on notes.arpanadhikari7.com.np

# Snoopy Bus Protocol

Two approaches are there to maintain cache coherency –

- Write-Invalidate : When a processor modifies its cache, all other cache copies are invalidated via the bus.
- Write-Update : This protocol demands the updated block content be broadcast to all cache copies via the bus.

## Write-invalidate



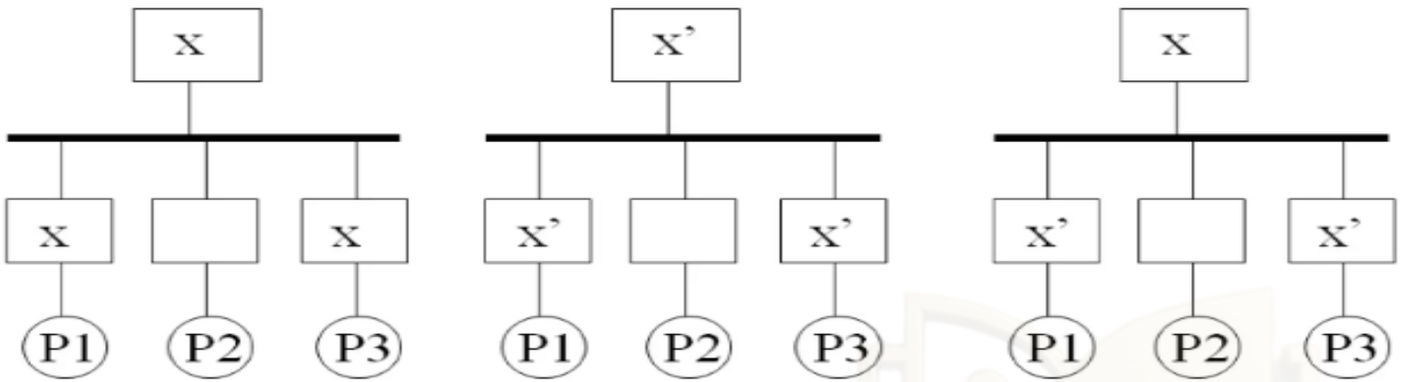

2 basic Write Invalidate snoopy protocols are-

**Write Invalidate - Write Through :** Multiple processors can read block copies from main memory safely until one processor updates its copy. At this time, all cache copies are invalidated and the main memory is updated to remain consistent.

**Write Invalidate - Write Back :** Whenever a shared block is updated, main memory updates are done only when the block is being replaced and all cache copies are invalidated.

Available on notes.arpanadhikari7.com.np

# Write-Update




Before Write Through Write back

2 basic Write Update snoopy protocols are-

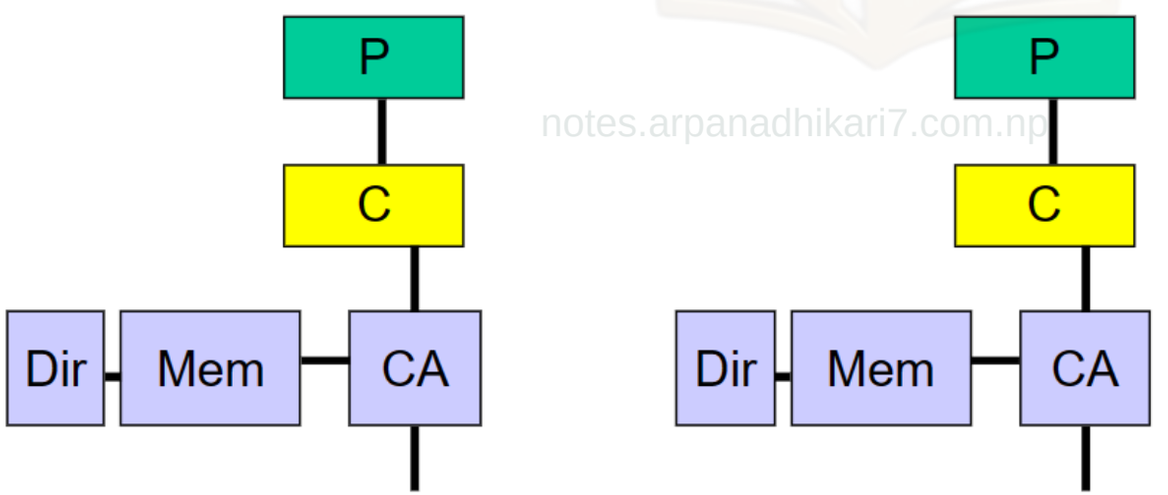
- Write Update – Write Through : Multiple processors can read block copies from main memory safely until one processor updates its copy. At this time, all cache copies are updated as well as the main memory is also updated to remain consistent.

- Write Update – Write Back : Whenever a shared block is updated, all cache copies are updated and main memory updates are done only when the block is being replaced .

Available on notes.arpanadhikari7.com.np

# Directory-Based Protocol

- For each block, there is a centralized “directory” that maintains the state of the block in different caches
- The directory is co-located with the corresponding memory
- Requests and replies on the interconnect are no longer seen by everyone – the directory serializes writes



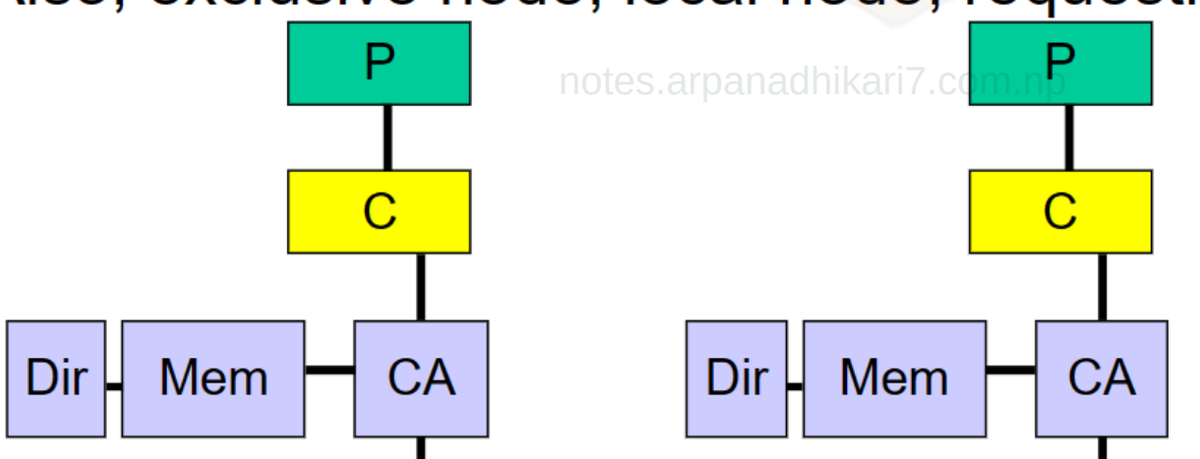

notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

# Definitions

- Home node: the node that stores memory and directory state for the cache block in question
- Dirty node: the node that has a cache copy in modified state
- Owner node: the node responsible for supplying data (usually either the home or dirty node)
- Also, exclusive node, local node, requesting node, etc.




notes.arpanadhikari7.com.np

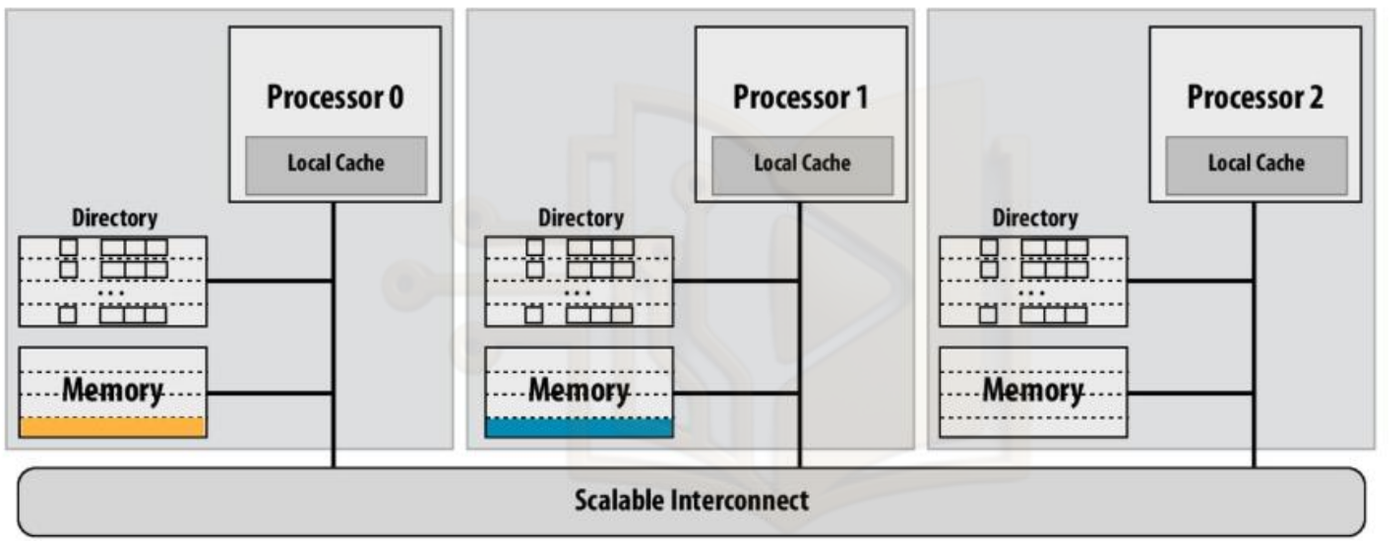


Available on notes.arpanadhikari7.com.np

# Directory-Based Cache Coherence

## A distributed directory

**Example: directory partition is co-located with memory it describes**




notes.arpanadhikari7.com.np

- **"Home node" of a line: node with memory holding the corresponding data for the line**
  **Example: node 0 is the home node of the yellow line, node 1 is the home node of the blue line**
- **"Requesting node": node containing processor requesting line**

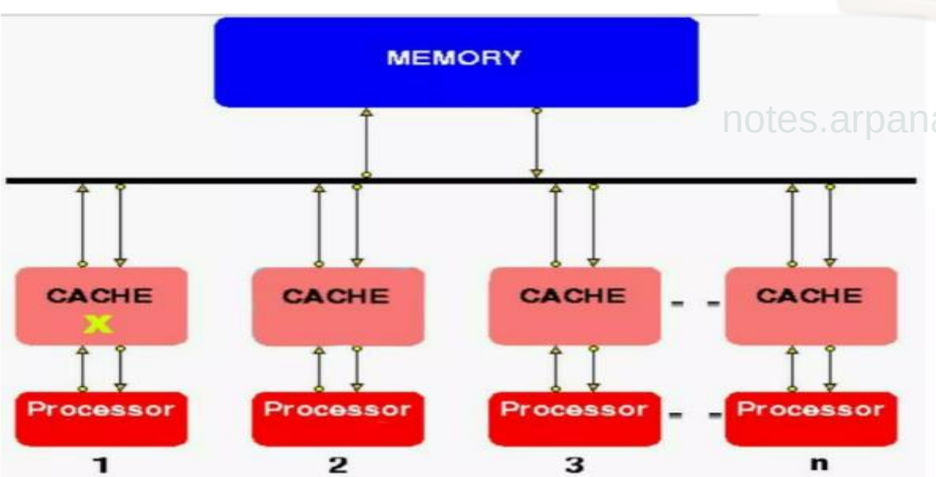
Available on notes.arpanadhikari7.com.np

# Protocol used by INTEL in Pentium to avoid Cache Coherence

- MESI protocol is used by INTEL in Pentium. It is an Invalidate based cache coherence protocol that supports Write-Back caches.

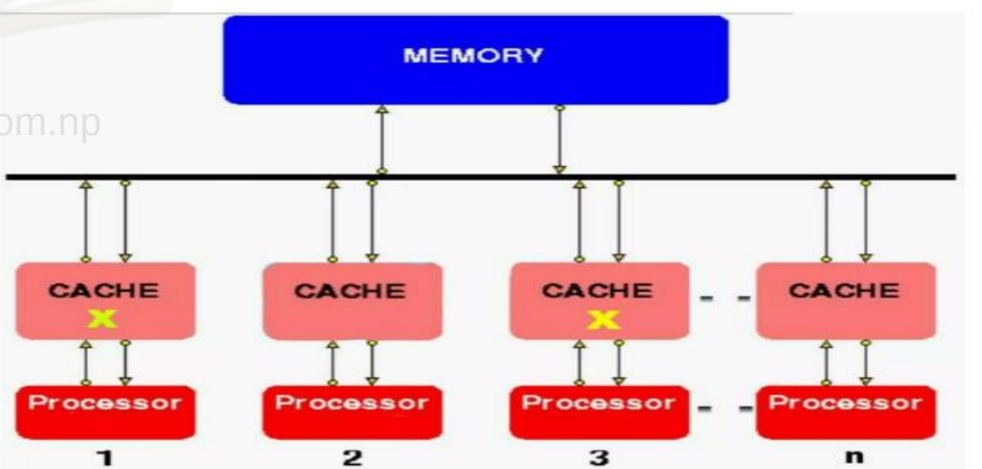
- The letters of the protocol name identify the possible states in which a cache can be -

**Exclusive** – Only Cache 1 has a copy X which is not modified.




- **M: Modified**
  - Only this cache has copy and is modified.
  - Main memory needs to be updated by this cache.
- **E: Exclusive**
  - Only this cache has copy which is not modified.
  - Main memory is up-to-date.
- **S: Shared**
  - More than one cache may have copies, which are not modified
  - Main memory is up-to-date
- **I: Invalid**
  - -- It indicates that cache is invalid.

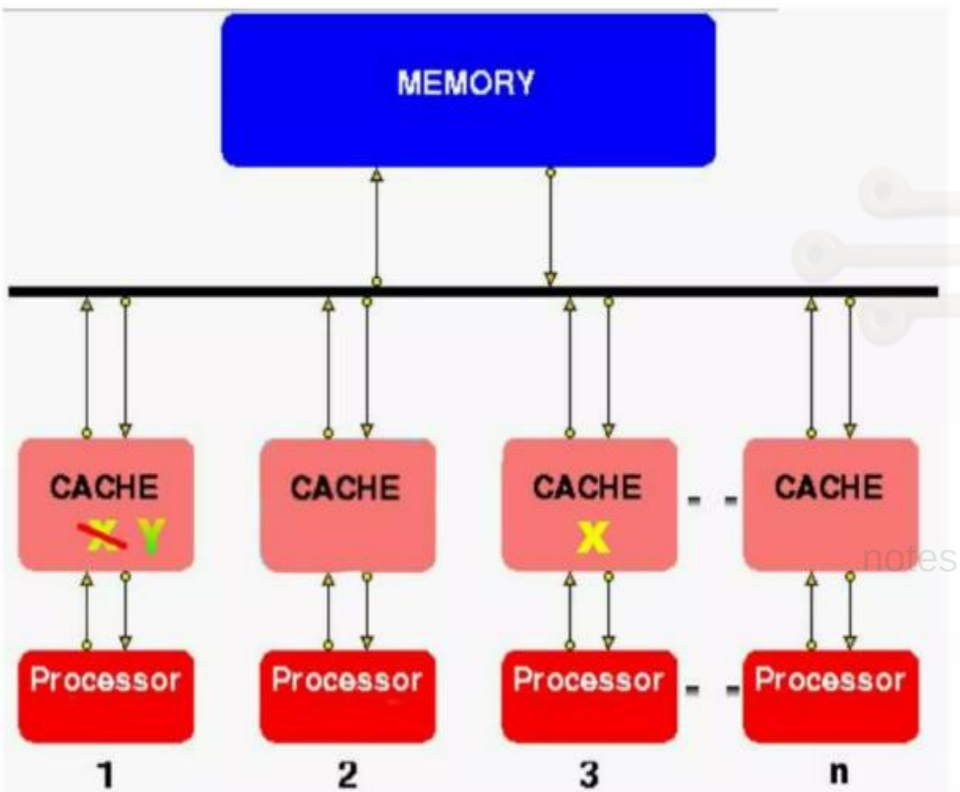
**Shared** — More than one cache may have a copy, here Cache 1and 3 are sharing a copy X



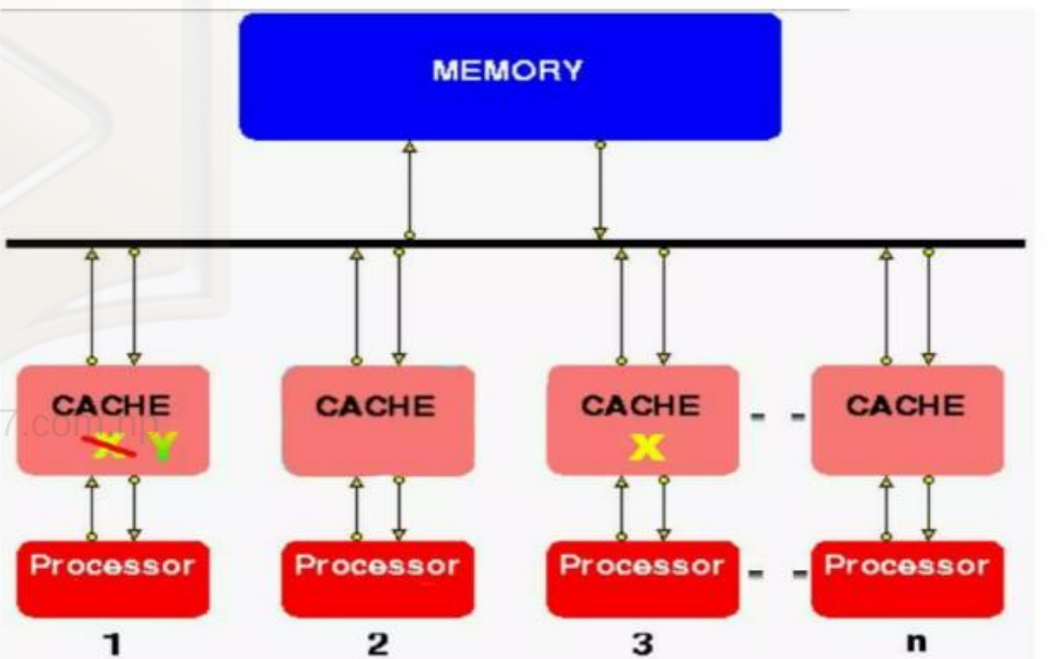

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Modified** – When the value at one local cache is modified. Example here Cache 1 is modified.




**Invalid** – When the value at one location is modified then that value in other cache is invalidated. Example here Cache 3's value is considered invalid.




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# MESI PROTOCOL EXAMPLE

- In this this example, a 4 processor shared memory system implements MESI Protocol.
- For following sequences of memory references, state of line containing variable X in each processor's cache is resolved.

| | P0's state | P1's state | P2's state | P3's state |
|---|---|---|---|---|
| Initial State | *I* | *I* | *I* | *I* |
| P0 reads X | *E* | *I* | *I* | *I* |
| P1 reads X | *S* | *S* | *I* | *I* |
| P2 reads X | *S* | *S* | *S* | *I* |
| P3 writes X | *I* | *I* | *I* | *M* |
| P0 reads X | *S* | *I* | *I* | *S* |

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Array

- A vector (one dimensional array)
- A set of vectors (multi-dimensional array)
- Array processor — a processor capable of processing array elements
- Suitable for scientific computations involving two dimensional matrices

# Array Processor

- Array processor performs a single instruction in multiple execution units in the same clock cycle
- The different execution units have same instruction using same set of vectors in the array

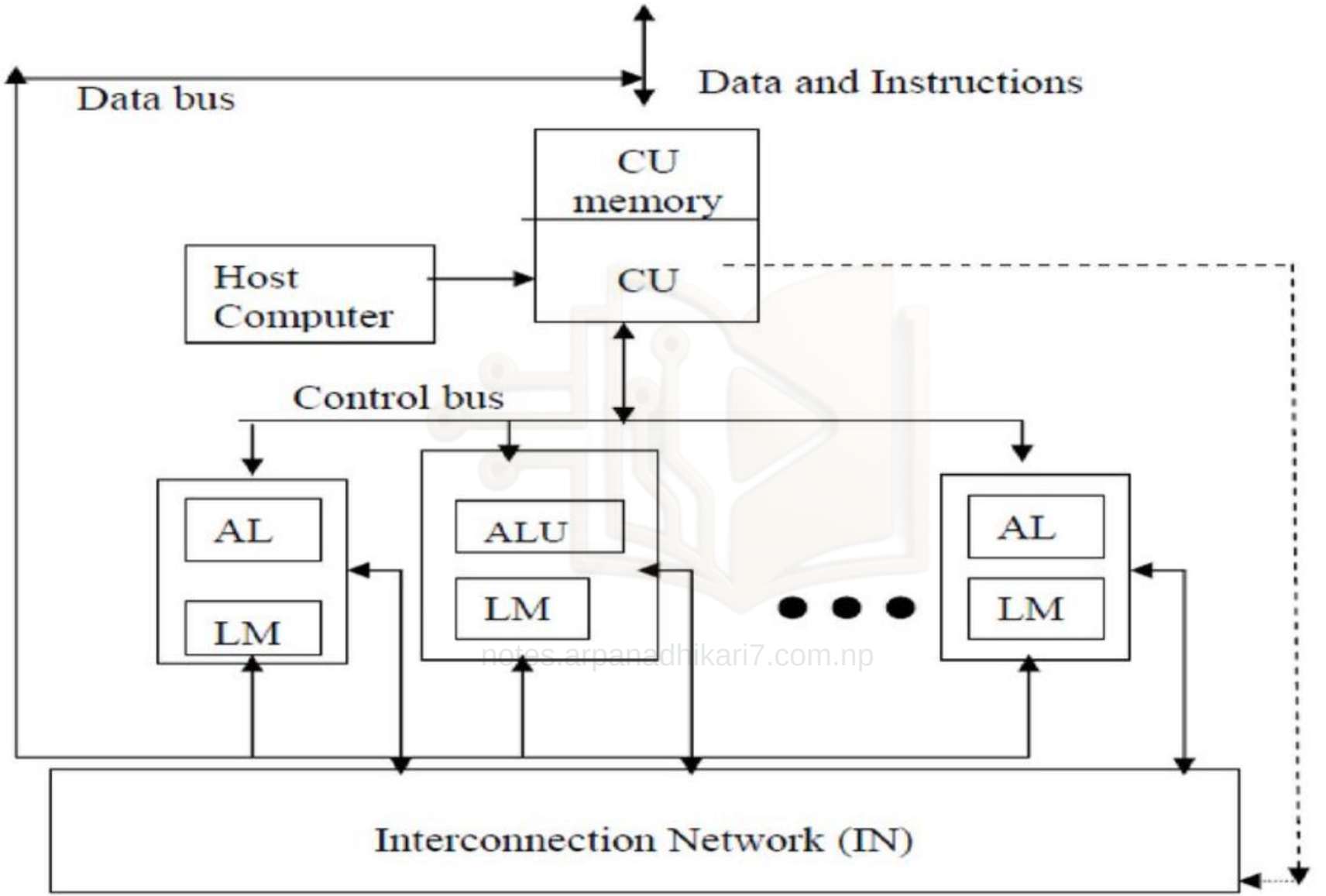
## Special features of Array processor

notes.arpanadhikari7.com.np

- Use of parallel execution units for processing different vectors of the arrays
- Use of memory interleaving, n memory address registers and n memory data registers in case of k pipelines and use of vector register files.

Available on notes.arpanadhikari7.com.np

# Organization of SIMD Array Processor




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# VECTOR PROGRAMMING

```
      DO  20  I = 1, 100
20    C(I) = B(I) + A(I)
```

Conventional computer

```
      Initialize I = 0
20    Read A(I)
      Read B(I)
      Store C(I) = A(I) + B(I)
      Increment I = i + 1
      If I ≤ 100 goto 20
```

Vector computer

```
      C(1:100) = A(1:100) + B(1:100)
```

notes.arpanadhikari7.com.np

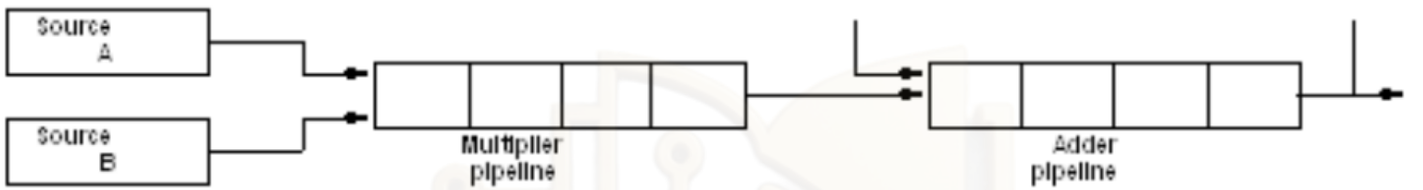
Available on notes.arpanadhikari7.com.np

# VECTOR INSTRUCTION FORMAT

## Vector Instruction Format

| Operation code | Base address source 1 | Base address source 2 | Base address destination | Vector length |
|---|---|---|---|---|

## Pipeline for Inner Product of Matrix Multiplication




$$C = A_1 B_1 + A_5 B_5 + A_9 B_9 + A_{13} B_{13} + \ldots\ldots + A_k B_k$$

K may be equal to 100 or even 1000

The values of A and B are either in memory or in processor registers. Each floating point adder and multiplier unit is supposed to have 4 segments. All segment registers are initially initialized to zero. Therefore the output of the adder is zero for the first 8 cycles until both the pipes are full.

Ai and Bi are brought in and multiplied at a rate of one pair per cycle. After 4 cycles the products are added to the Output of the adder. During the next 4 cycles zero is added. At the end of the 8 $^{th}$ cycle the first four products A1B1 through A4B4 are in the four adder segments and the next four products A5 B5 through A8B8 are in the multiplier Segments.
Thus the 9 $^{th}$ cycle and onwards starts breaking down the summation into four sections:

Available on notes.arpanadhikari7.com.np

- $C = A_1 B_1 + A_5 B_5 + A_9 B_9 + A_{13} B_{13} + \ldots\ldots$

$+ A_2 B_2 + A_6 B_6 + A_{10} B_{10} + A_{14} B_{14} + \ldots\ldots$

$+ A_3 B_3 + A_7 B_7 + A_{11} B_{11} + A_{15} B_{15} + \ldots..$

$+ A_4 B_4 + A_8 B_8 + A_{12} B_{12} + A_{16} B_{16} + \ldots.$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# VECTOR PROCESSING

Vector Processing Applications

- Problems that can be efficiently formulated in terms of vectors
  - Long-range weather forecasting
  - Petroleum explorations
  - Seismic data analysis
  - Medical diagnosis
  - Aerodynamics and space flight simulations
  - Artificial intelligence and expert systems
  - Mapping the human genome
  - Image processing

Vector Processor (computer)

Ability to process vectors, and related data structures such as matrices and multi-dimensional arrays, much faster than conventional computers

Vector Processors may also be pipelined

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Basic Example of Scalar and Vector Processing

For example, consider the following vector addition:

$$\mathbf{A + B \longrightarrow C}$$

The vectors are of length 500. This operation through the sequential computer can be specified by 500 add instructions as given below:

$$C[1] = A[1] + B[1]$$
$$C[2] = A[2] + B[2]$$
$$\text{---------- ---------- ----------}$$
$$C[500] = A[500] + B[500]$$

If we perform the same operation through a pipelined-vector computer then it does not break the vectors in 500 add statements. Because a vector processor has the set of vector instructions that allow the operations to be specified in one vector instruction as:

$$\mathbf{A\ (1:500) + B\ (1:500) \longrightarrow C\ (1:500)}$$

Each vector operation may be broken internally in scalar operations but they are executed in parallel which results in mush faster execution as compared to sequential computer.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Threads



- **A thread is a basic unit of CPU utilization.**
- **A *thread* is a separate process** with **its own instructions** and **data.**
- **A thread may represent a process** that is **part of a parallel program consisting of multiple processes**, or **it may represent an independent program.**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Threads



- It comprises of a **thread ID,** a **program counter,** a **register set** and a **stack.**

- **It shares** its **code section, data section,** and **other operating-system resources,** such as open files and signals with **other threads belonging to the same Process**.

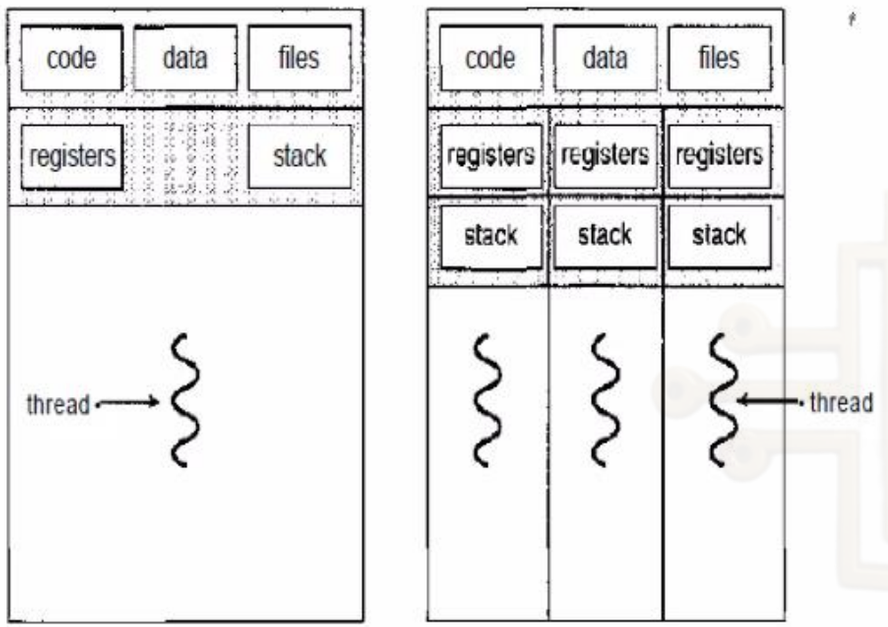
- **A traditional process has a single thread of control.** If a process has multiple threads of control, it can perform more than one task at a time.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Threads






single-threaded process

multithreaded process

- **Many software packages** that run on modern desktop PCs **are multithreaded.**
- **For example:**

  A word processor may have:

  a thread for displaying graphics,

  another thread for responding to keystrokes from the user, and

  a third thread for performing spelling and grammar checking in the background.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Multithreading



- **Benefits:**

1. **Responsiveness:** Multithreading is an **interactive application that may allow a program to continue running even if part of it is blocked,** thereby increasing responsiveness to the user.

   For example: A multithreaded web browser could still allow user interaction in one thread while an image was being loaded in another thread.

2. **Resource sharing:** By default, **threads share the memory and the resources of the process** to which they belong. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Multithreading



- **Benefits:**

3. **Economy:** Allocating memory and resources for process creation is costly. **Since threads share resources of the process** to which they belong, **they will provide cost effective solution.**

4. **Utilization of multiprocessor architectures:** In multiprocessor architecture, threads may be running in parallel on different processors.

   A single threaded process can only run on one CPU, no matter how many are available.

   Multithreading on a multi-CPU machine increases concurrency.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Multithreading Models



- **Support for threads** may be **provided either at the user level** or **at the kernel level.**

- **User threads are supported above the kernel** and are managed without kernel support, whereas **kernel threads are supported and managed directly by the operating system.**

notes.arpanadhikari7.com.np

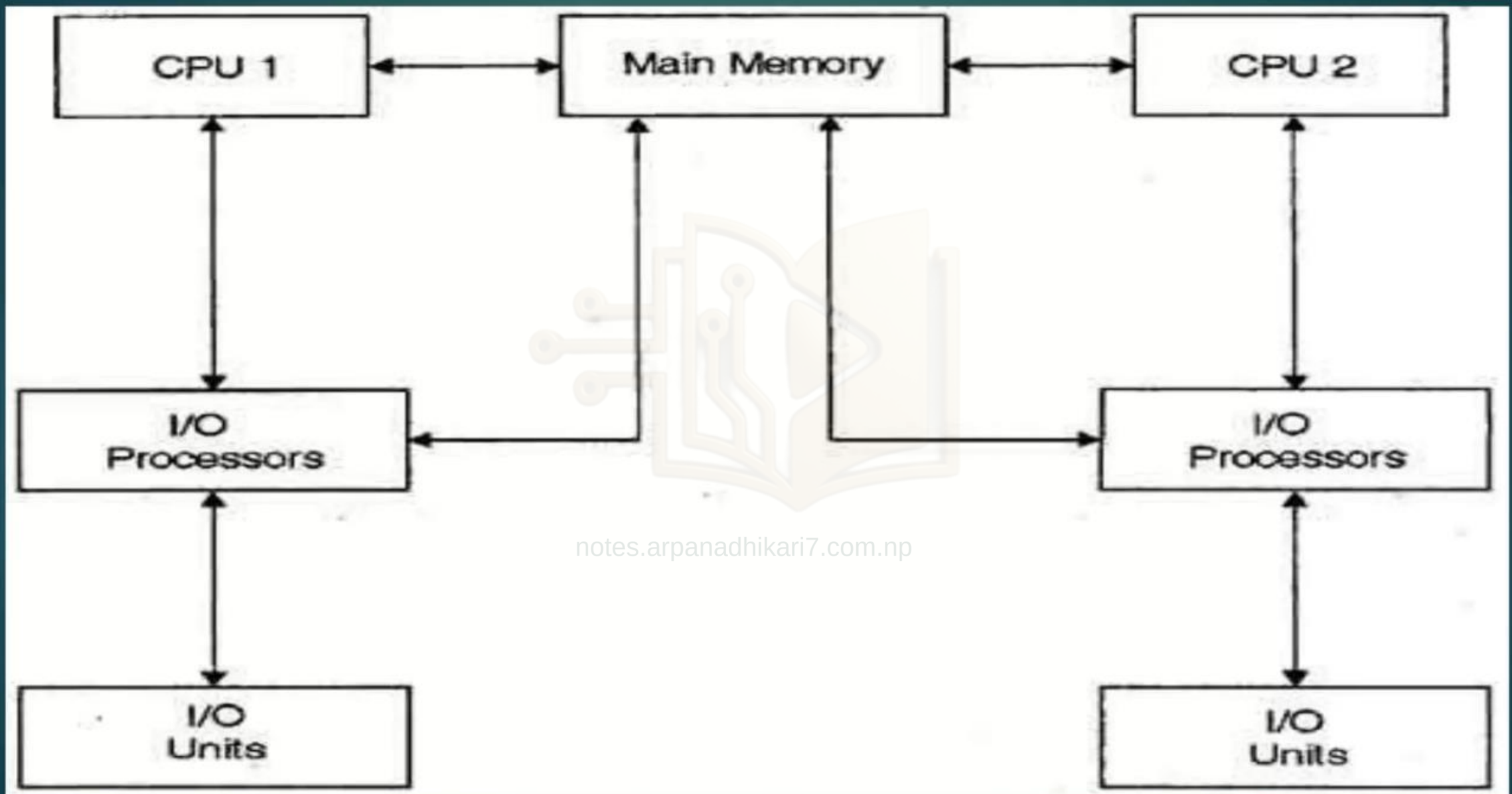
Available on notes.arpanadhikari7.com.np

# INTRODUCTION

- Multiprocessor system is an interconnection of two or more CPUs, each sharing main memory and peripherals , in order to simultaneously process programs.
- Multiprocessing improves the reliability of the system, which means if a fault causes one processor to fail , a second processor can be assigned to perform the function of the disabled processor. Thus, a system can continue to function correctly.
- The main benefit derived from a multiprocessor organization is an improved system performance.
- The components that forms multiprocessor are CPUs , input-output devices and memory unit shown below in a diagram...

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Why Choose a Multiprocessor?

- A single CPU can only go so fast, use more than one CPU to improve performance.
- Multiple users
- Multiple applications
- Multi-tasking within an application
- Responsiveness and/or throughput
- Share hardware between CPUs

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# How Multiprocessor Are Classified?

- Multiprocessor are classified by the way their memory is organised, mainly it is classified into two types, which include:
  1. Tightly Coupled Multiprocessor.
  2. Loosely Coupled Multiprocessor.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Tightly Coupled Multiprocessor

- A multiprocessor system with common shared memory is classified as a shared-memory or tightly coupled multiprocessor.
- Most commercial tightly coupled multiprocessors provide a cache memory with each CPU.
- In addition, there is a global common memory that all CPUs can access.
- Information can therefore be shared among the CPUs by placing it in the common global memory.
- Tightly coupled systems are most efficient when the interaction between tasks is higher.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Loosely Coupled Multiprocessor.

- An alternative model of microprocessor is the distributed-memory or loosely coupled system. Each processor element in a loosely coupled system has its own private local memory.
- The processors are tied together by a switching scheme designed to route information from one processor to another through a message passing scheme.
- The processors relay program and data to other processors in packets.
- A packet consists of an address , the data content and some error detection code.
- Loosely coupled systems are most efficient when the interaction between tasks is minimal.

Available on notes.arpanadhikari7.com.np

# INTERCONNECTION STRUCTURES

- The physical forms for establishing an interconnection network include:
  - ✓ **BUS-ORIENTED SYSTEMS.**
  - ✓ **MULTIPORT MEMORY.**
  - ✓ **CROSSBAR-CONNECTION SYSTEMS.**
  - ✓ **MULTISTAGE SWITCHING NETWORK.**
  - ✓ **HYPERCUBE SYSTEM.**

Available on notes.arpanadhikari7.com.np

# 1. BUS-ORIENTED SYSTEM

- It is one of the simplest way and uses a shared bus to connect various processors and memories.
- A large number of processors can communicate with each other and with the globally shared memory over a shared bus.
- In a shared bus organisation ,there are main two points of contention . These are the shared bus itself and the shared memory.
- In order to reduce the contention, cache memory is used in two different arrangements . These include:
  - ✓ Cache can be associated with the shared memory and processor access it over the bus.
  - ✓ Cache can be associated with each individual processor.

Available on notes.arpanadhikari7.com.np

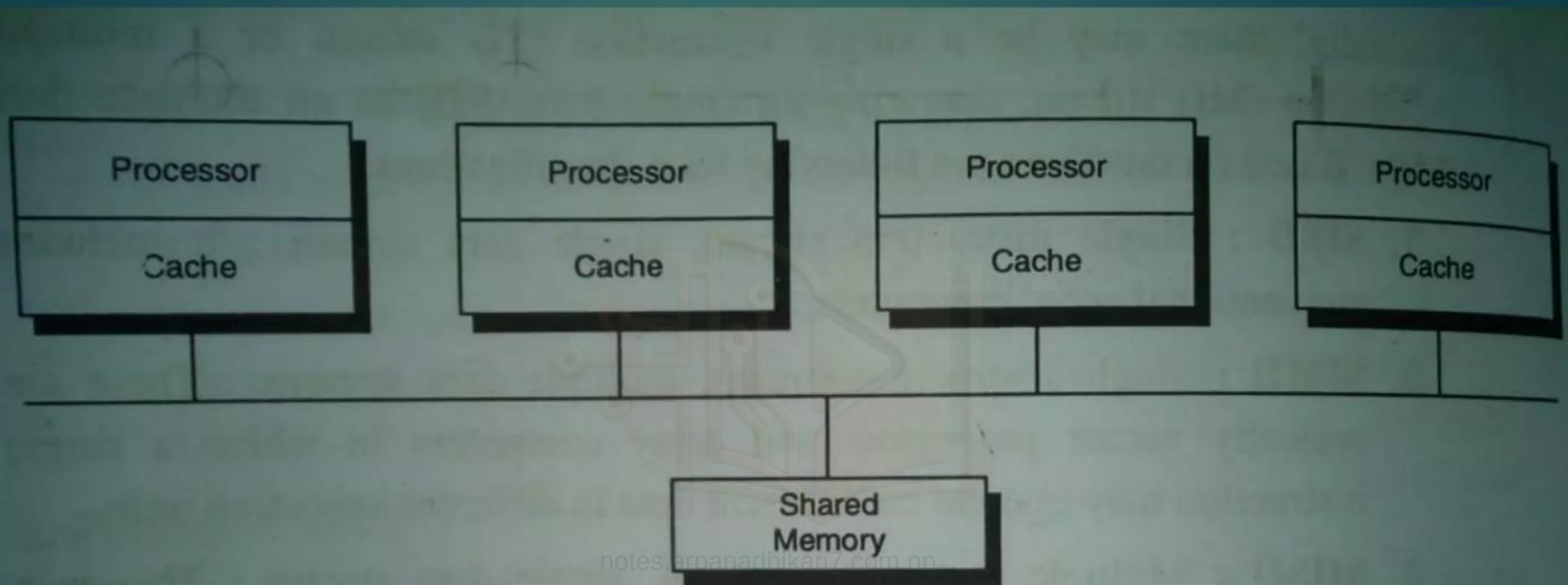



**Fig. 12.1** Shared bus multiprocessor organization

notes.arpanadhikari7.com.np

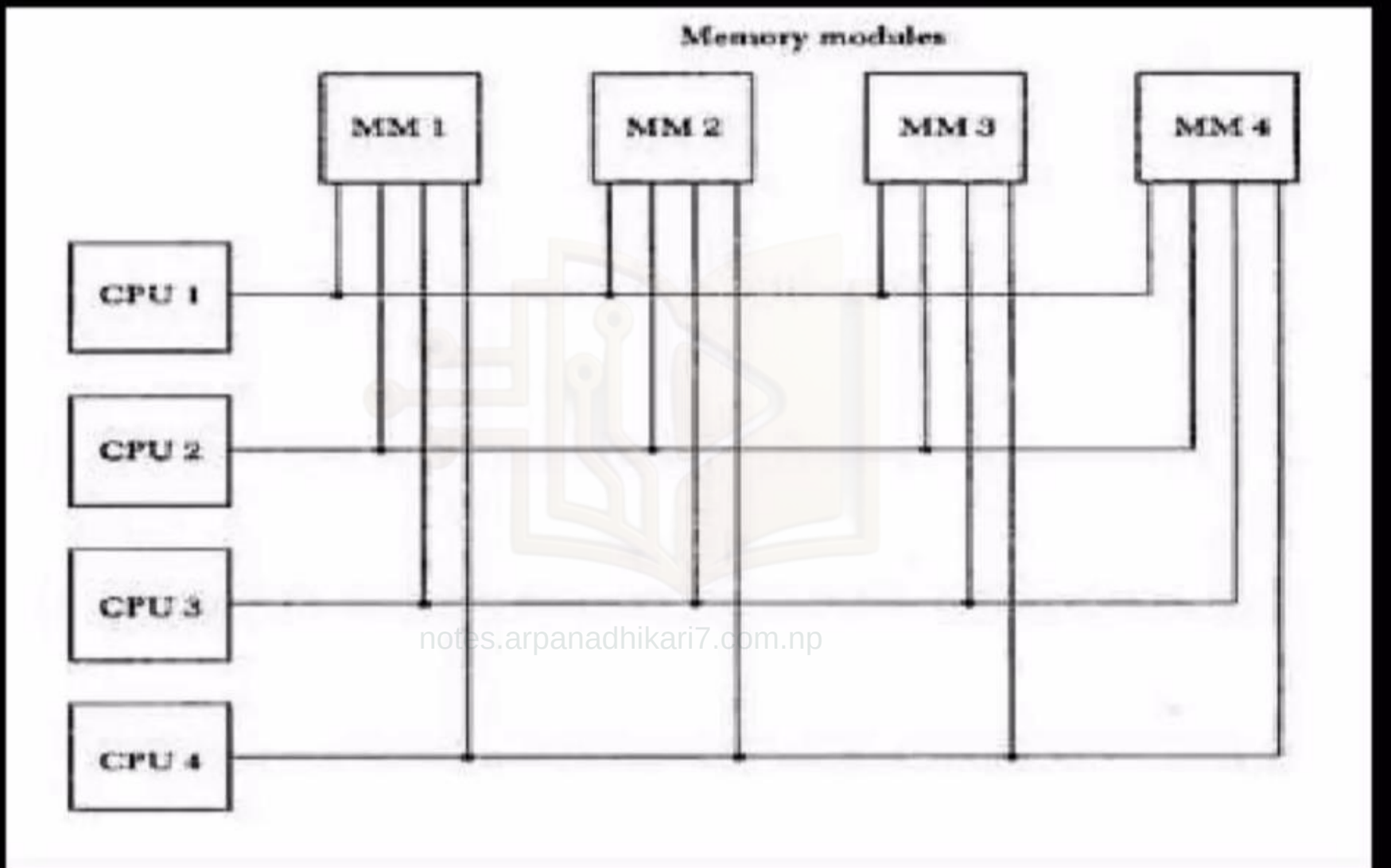
Available on notes.arpanadhikari7.com.np

# 2.MULTIPORT MEMORY

- A multiport memory system employs separate buses between each memory module and each CPU.
- The module must have internal control logic to determine which port will have access to memory at a given time.
- Memory access conflicts are resolved by assigning fixed priorities to each memory port.

| ADVANTAGE | DISADVANTAGE |
| --- | --- |
| High transfer rate can be achieved because of multiple paths. | It requires expensive memory control logic and a large number of cables and connections. So good for systems with small number of processors. |

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

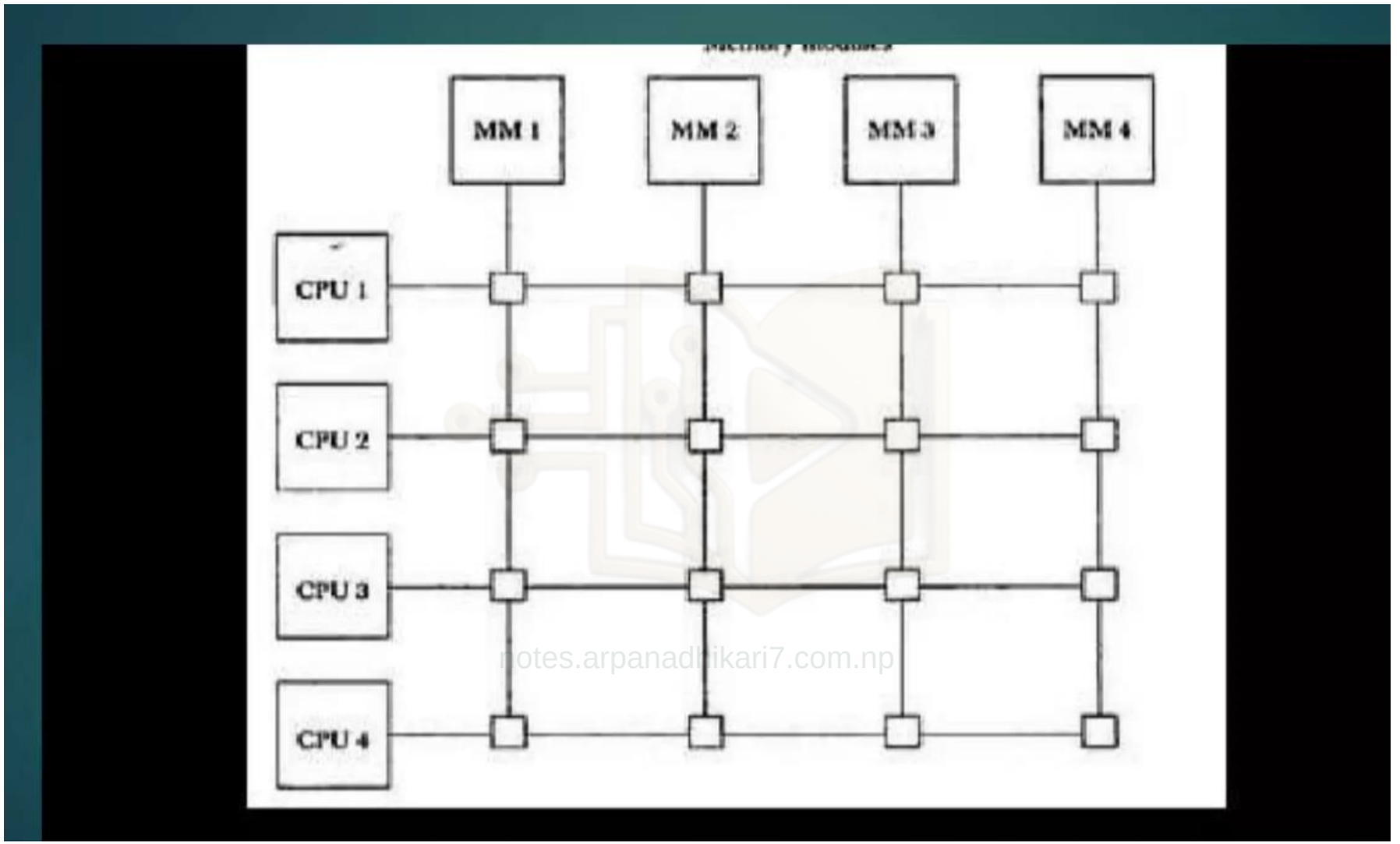
# 3. CROSSBAR CONNECTED SYSTEM

- The crossbar represents the antithesis of the bus.
- It allows simultaneous access of n processors to n memories , provided that each processor accesses a different memory .Thus , it is free from contention problem(occur only when more than one processor tries to access the same memory at the same time).
- The small square in each cross-point is a switch that determines the path from a processor to a memory module.

| ADVANTAGE | DISADVANTAGE |
| --- | --- |
| Supports simultaneous transfers from all memory modules. | The hardware required to implement the switch can become quite large and complex. ($n^2$ cross-points to connect n processors to n memories ). |

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



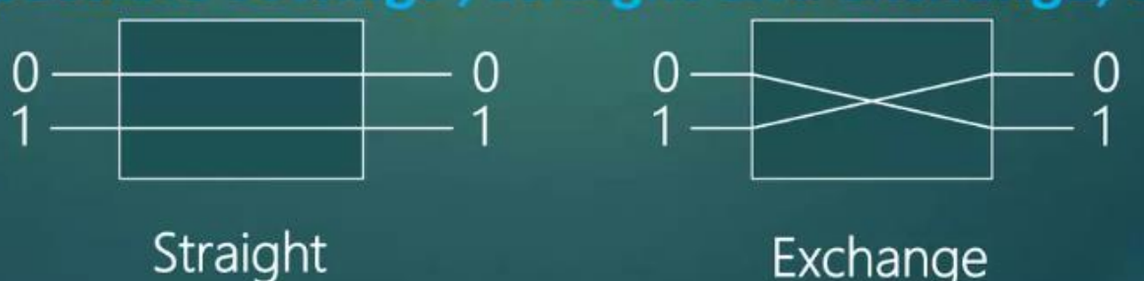

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# 4. MULTISTAGE SWITCHING NETWORK

- Multistage interconnection network is a high-speed computer network usually composed of processing elements on one end and memory elements on the other end of the network connected by switching elements.
- A generalized form of multistage switch provides links between n inputs and n outputs.
- The switching network can connect any input to any output by making appropriate connections in each of the stages.
- Routing in this is usually fixed.
- 2 x 2 switches have 2 possible settings , straight and exchange, as shown in figures below

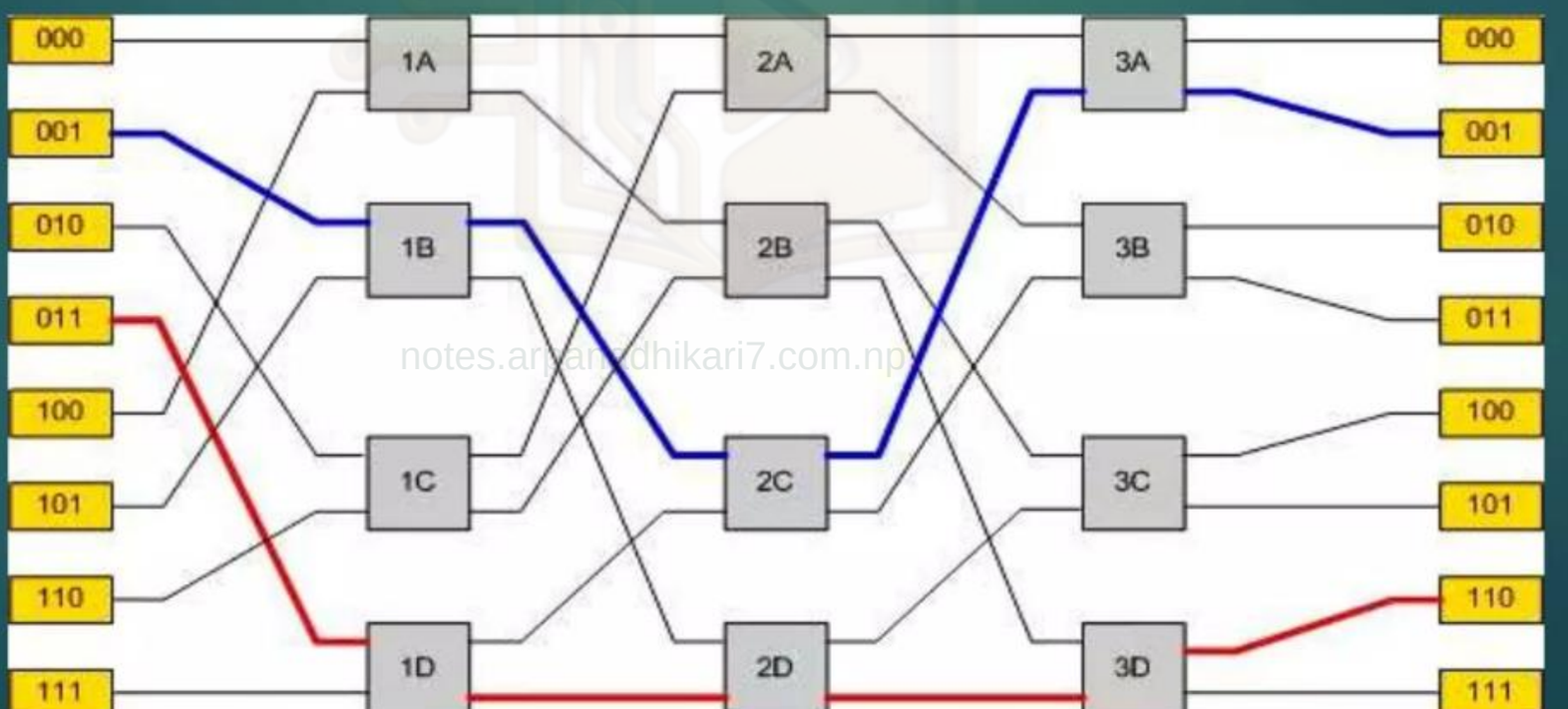
notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np

- Some widely used multistage interconnection networks include:
  - Clos network
  - Benes network
  - Omega network
  - Baseline network

-Figure below shows the structure of an omega network using n CPUs and n memories.




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- -The figure illustrates how 2 CPUs A(001) and B(011) accesses two different memory blocks M(001) and N(110).

  -If any other CPU wants to access the memory 001,it has to wait.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# 5. HYPERCUBE INTERCONNECTION

- The hypercube or binary n-cube multiprocessor structure is composed of $N=2^n$ processor interconnected in an n-dimensional binary cube.
- Each processor forms a node of the cube containing not only a CPU but also local memory and I/0 interface .
- Each processor address differs from that of each of its n neighbours by exactly one bit position.

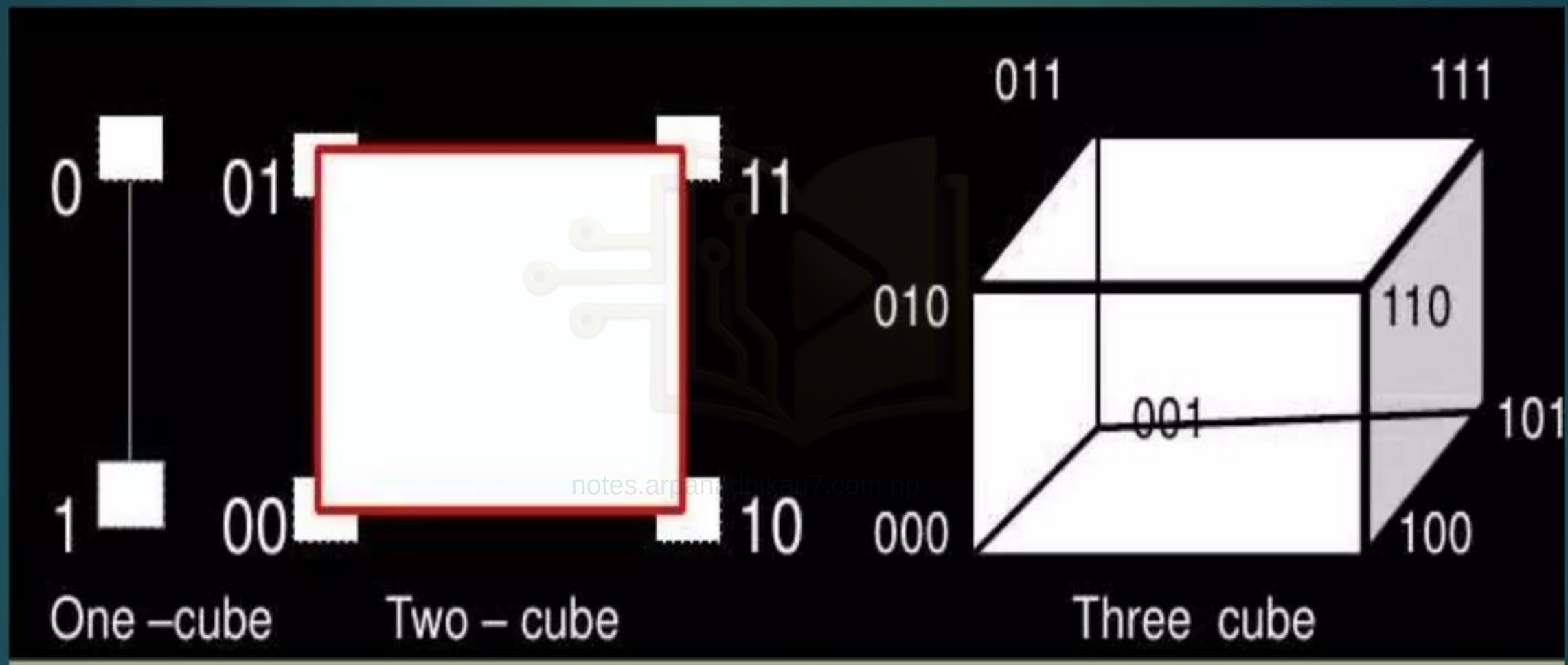
notes.arpanadhikari7.com.np

-ADVANTAGES

- It incurs less cost.
- The communication between adjacent nodes is direct.

Available on notes.arpanadhikari7.com.np

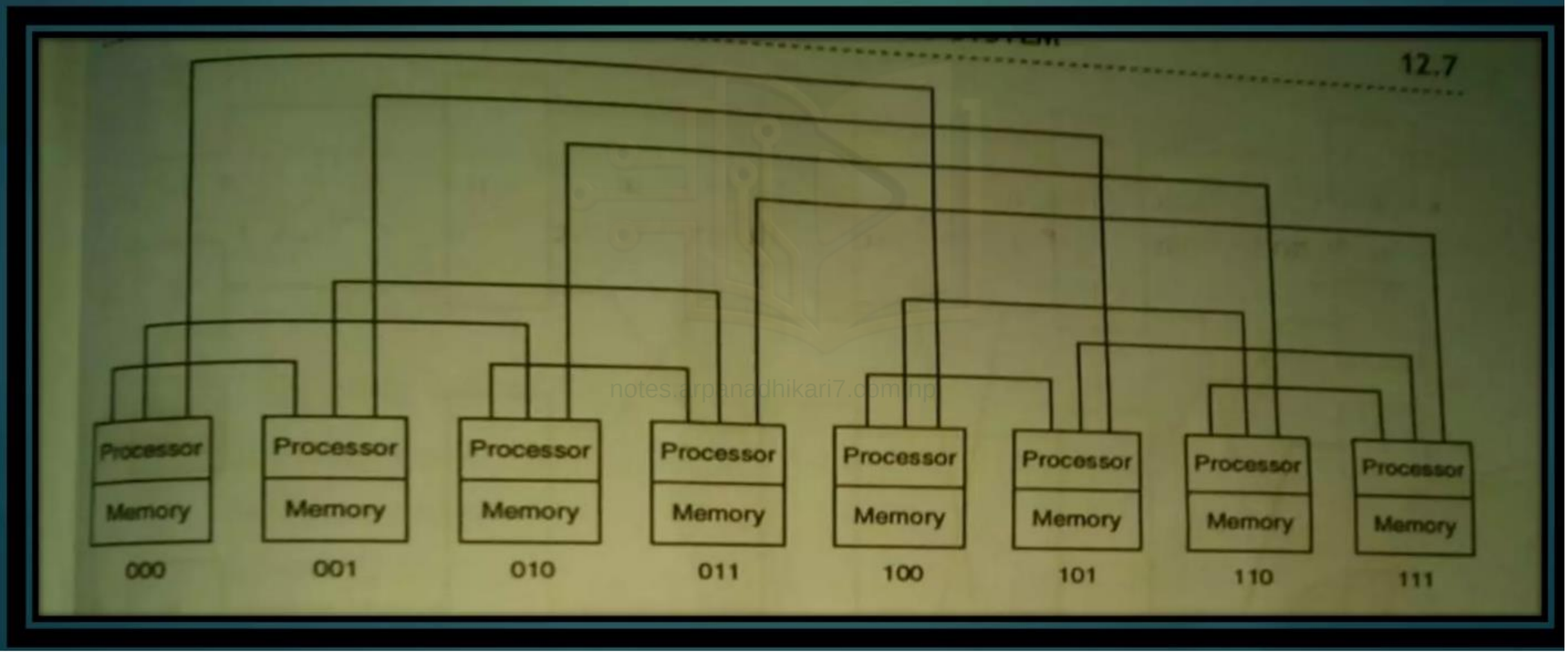
# Hyper cube structure for n=1,2,3.




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Each processor in a hypercube has direct physical link to log2n other nodes in a n-node system.
  For example , if N=8 then each node has direct links to $\log_2 8=3$ other nodes as shown in the figure below:




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np